



Chuyên đề CNPM - aaaaaaaaaaaaaaa

Học viện công nghệ bưu chính viễn thông (Học viện Công nghệ Bưu chính Viễn thông)



Scan to open on Studeersnel

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO
CHUYÊN ĐỀ CÔNG NGHỆ PHẦN MỀM
CÁC THUẬT TOÁN ĐỐI SÁNH MẪU
(PATTERN MATCHING ALGORITHMS)

SINH VIÊN THỰC HIỆN: NGUYỄN BÁ NHẬT
MÃ SV: B17DCCN479
NHÓM: 04
GIẢNG VIÊN: NGUYỄN DUY PHƯƠNG

Hà Nội, 5/2021

MỤC LỤC

I. TỔNG QUAN: SẮP XẾP CÁC THUẬT TOÁN THÀNH 4 LOẠI.....	3
II. TÌM KIẾM MẪU TỪ TRÁI SANG PHẢI.....	3
2.1. Thuật toán Brute Force	3
2.2. Search with an automaton.....	6
2.3. Thuật toán Karp-Rabin	9
2.4. Thuật toán Shift Or	13
2.5. Thuật toán Morris-Pratt	15
2.6. Thuật toán Knuth-Morris-Pratt.	18
2.7. Thuật toán Apostolico-Crochemore.....	21
2.8. Thuật toán Not So Naïve	25
III. TÌM KIẾM MẪU TỪ PHẢI SANG TRÁI.....	29
3.1. Thuật toán Boyer-Moore	29
3.2. Thuật toán Turbo-BM.....	32
3.3. Thuật toán Quick Search.....	35
3.4. Thuật toán Tuned-Boyer-Moore.....	37
3.5. Thuật toán Zhu-Takaoka.....	39
3.6. Thuật toán Berry-Ravindran	42
3.7. Thuật toán Apostolico-Giancarlo.....	45
IV. TÌM KIẾM MẪU TỪ VỊ TRÍ XÁC ĐỊNH.....	49
4.1. Thuật toán Colussi	49
4.2. Thuật toán Skip Search	54
4.3. Thuật toán Alpha Skip Search	55
V. TÌM KIẾM MẪU TỪ VỊ TRÍ BẤT KỲ	59
5.1. Thuật toán Horspool algorithm.....	59
5.2. Thuật toán Smith.....	61
5.3. Thuật toán Raita.....	63

I. TỔNG QUAN: SẮP XẾP CÁC THUẬT TOÁN THÀNH 4 LOẠI

Các thuật toán tìm kiếm mẫu từ trái sang phải gồm:

- Thuật toán Brute Force
- Search with an automaton
- Thuật toán Karp-Rabin
- Thuật toán Shift Or
- Thuật toán Morris-Pratt
- Thuật toán Knuth-Morris-Pratt
- Thuật toán Apostolico-Crochemore
- Thuật toán Not So Naïve

Các thuật toán tìm kiếm mẫu từ phải sang trái gồm:

- Thuật toán Boyer-Moore
- Thuật toán Turbo-BM
- Thuật toán Quick Search
- Thuật toán Tuned-Boyer-Moore
- Thuật toán Zhu-Takaoka
- Thuật toán Berry-Ravindran
- Thuật toán Apostolico-Giancarlo

Các thuật toán tìm kiếm mẫu từ vị trí xác định gồm:

- Thuật toán Colussi
- Thuật toán Skip Search
- Thuật toán Alpha Skip Search

Các thuật toán tìm kiếm mẫu từ vị trí bất kỳ gồm:

- Thuật toán Hospool
- Thuật toán Smith
- Thuật toán Raita

II. TÌM KIẾM MẪU TỪ TRÁI SANG PHẢI

2.1. Thuật toán Brute Force

Trình bày thuật toán:

Thuật toán Brute Force kiểm tra tất cả vị trí trong đoạn text từ vị trí 0 đến $n - m$, xem liệu mẫu (pattern) có khớp ngay vị trí bắt đầu không. Sau mỗi lần kiểm tra, nó dịch mẫu sang phải 1 vị trí. Đặc điểm: không cần pha tiền xử lý, bộ nhớ cần dùng cố định.

Đánh giá độ phức tạp thuật toán: $O(m \times n)$

Kiểm nghiệm thuật toán:

Giả sử ta có 2 mảng ký tự y và x . Ta tiến hành tìm mảng x trong mảng y .

Lần 1:

y G C A T C G C A G A G A G T A T A C A G T A C G
1 2 3 4
 x G C A G A G A G

Lần 2:

y G C A T C G C A G A G A G T A T A C A G T A C G
1
 x G C A G A G A G

Lần 3:

y G C A T C G C A G A G A G T A T A C A G T A C G
1
 x G C A G A G A G

Lần 4:

y G C A T C G C A G A G A G T A T A C A G T A C G
1
 x G C A G A G A G

Lần 5:

y G C A T C G C A G A G A G T A T A C A G T A C G
1
 x G C A G A G A G

Lần 6:

y G C A T C G C A G A G A G T A T A C A G T A C G
1 2 3 4 5 6 7 8
 x G C A G A G A G

Lần 7:

y G C A T C G C A G A G A G T A T A C A G T A C G
1
 x G C A G A G A G

Lần 8:

Lần 9:

Lần 10:

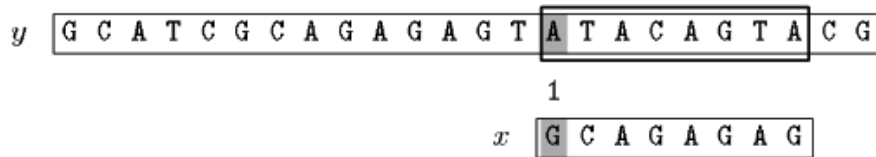
Lần 11:

Lần 12:

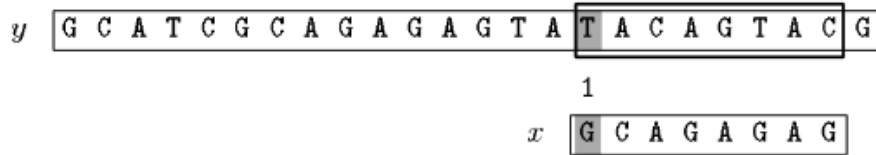
Lần 13:

Lần 14:

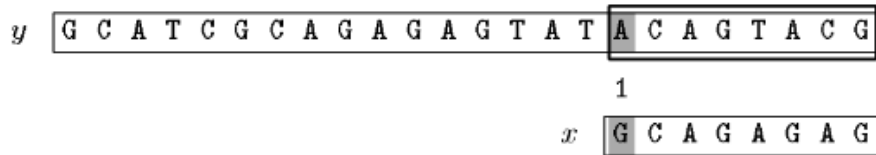
Lần 15:



Lần 16:



Lần 17:



Vậy ta thấy chỉ có lần 6 là tìm thấy mảng x trong mảng y.

Trong ví dụ trên, thuật toán Brute Force phải thực hiện 30 lần so sánh.

Lập trình theo thuật toán:

```
void BF(char *x, int m, char *y, int n){
    int i, j;

    for(j = 0; j <= n - m; ++j){
        for(i = 0; i < m && x[i] == y[i + j]; ++i);

        if(i >= m)
            printf("%d \n", j);
    }
}
```

2.2. Search with an automaton

Trình bày thuật toán:

Thuật toán này xây dựng DFA (viết tắt của Deterministic Finite Automaton – máy trạng thái hữu hạn) giúp tìm kiếm mẫu trên văn bản một cách nhanh chóng.

Định nghĩa máy trạng thái hữu hạn DFA:

DFA là một bộ gồm $(Q, \Sigma, \delta : Q \times \Sigma \rightarrow Q, q_0 \in Q, F \subseteq Q)$. Trong đó:

- Q – tập tất cả các tiền tố của mẫu $x = x_0 \dots x_{m-1}$:
 $Q = \{\epsilon, x_0, x_0x_1, x_0x_1x_2, \dots, x_0x_1 \dots x_{m-1}\}$
- $q_0 = \epsilon$ – trạng thái biểu diễn tiền tố rỗng.
- $F = x$ – trạng thái biểu diễn tiền tố trùng với mẫu x.
- Σ là tập các ký tự có trong mẫu x và văn bản y.

- δ – một hàm từ $Q \times \Sigma$ vào Q , gọi là hàm chuyển tiếp (transition function).
 - Đối với mỗi $q \in Q$ và $c \in \Sigma$ thì $\delta(q, c) = qc$ khi và chỉ khi $qc \in Q$.
 - Ngược lại $\delta(q, c) = p$ sao cho p là hậu tố dài nhất của qc và p cũng là tiền tố của x ($p \in Q$).

Trong thuật toán này, quá trình tìm kiếm được đưa về một quá trình biến đổi trạng thái automat. Hệ thống automat trong thuật toán DFA sẽ được xây dựng dựa trên mẫu. Mỗi trạng thái của automat sẽ đại diện cho số ký tự đang khớp của mẫu với văn bản. Trạng thái ban đầu được gán bằng q_0 . Các ký tự của văn bản sẽ làm thay đổi trạng thái. Và khi đạt được trạng thái cuối cùng F có nghĩa là đã tìm được một vị trí xuất hiện của mẫu.

Đánh giá độ phức tạp thuật toán:

- Pha tiền xử lý có độ phức tạp là: $O(m \times \sigma)$
- Pha tìm kiếm có độ phức tạp là $O(n)$ nếu DFA được chứa trong bảng truy cập trực tiếp, ngược lại có độ phức tạp là $O(n \times \log \sigma)$.

Kiểm nghiệm thuật toán:

Giả sử ta tìm kiếm mẫu $x = abaa$ trong văn bản $y = ababbaabaaab$.

Pha tiền xử lý:

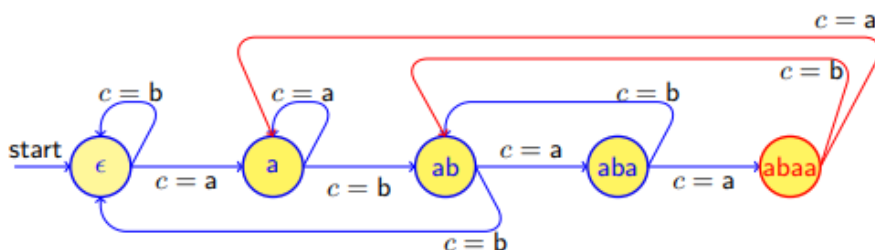
Ta tính được DFA:

$\Sigma = \{a, b\}$; $Q = \{\epsilon, a, ab, aba, abaa\}$; $q_0 = \epsilon$; $F = abaa$

Hàm chuyển tiếp

$\delta(q, c)$:

$c \backslash q$	ϵ	a	ab	aba	abaa
a	a	a	aba	abaa	a
b	ϵ	ab	ϵ	ab	ab



$$q_{\text{next}} = \delta(q_{\text{curr}}, c) =$$

$q_{\text{curr}} \backslash c$	a	b
ϵ	a	ϵ
a	a	ab
ab	aba	ϵ
aba	abaa	ab
abaa	a	ab

Pha tìm kiếm:

STEP	TEXT	TRANSITION	
1	ababbaabaaab	ε	→ a
2	a a babbaabaaab	a	→ ab
3	ab a babbaabaaab	ab	→ aba
4	abab a babbaabaaab	aba	→ ab
5	ababba a abaaab	ab	→ ε
6	ababbaab a aaab	ε	→ a
7	ababbaaba a aaab	a	→ a
8	ababbaabab a aaab	a	→ ab
9	ababbaababa a aaab	ab	→ aba
10	ababbaababab a aaab	aba	→ abaa
11	ababbaababab a ab	abaa	→ a
12	ababbaababab a ab	a	→ ab

Như vậy ta thấy tại bước thứ 11, thuật toán đã tìm thấy một mẫu x trong văn bản y.

Lập trình theo thuật toán:

```
#include<stdio.h>
#include<string.h>
#define MAX 256

int getNextState(char *pat, int m, int state, int x);
void computeTF(char *pat, int m, int TF[][MAX]);
void finite_automata(char *pat, char *txt);

int main()
{
    char txt[1000], pat[1000];
    printf("Input txt: ");
    gets(txt);
    printf("Input pat: ");
    gets(pat);
    finite_automata(pat, txt);
    return 0;
}

int getNextState(char *pat, int m, int state, int x)
{
    int ns, i;
    if(state < m && x == pat[state])
        return state + 1;
    for(ns = state; ns > 0; ns--)
    {
        if(pat[ns - 1] == x)
        {
            for(i = 0; i < ns - 1; i++)
            {
                if(pat[i] != pat[state - ns + 1 + i])
                    break;
            }
        }
    }
}
```

```

        if(i == ns - 1)
            return ns;
    }
}
return 0;
}

void computeTF(char *pat, int m, int TF[][MAX])
{
    int state, x;
    for(state = 0; state <= m; state++)
        for(x = 0; x < MAX; x++)
            TF[state][x] = getNextState(pat, m, state, x);
}

void finite_automata(char *pat, char *txt)
{
    int i, count = 0, state = 0;
    int m, n;
    m = strlen(pat);
    n = strlen(txt);
    int TF[m+1][MAX];
    computeTF(pat, m, TF);
    for(i = 0; i <= n; i++)
    {
        state=TF[state][txt[i]];
        if(state>=m)
            printf("Position: %d\n", (i - (m - 1)));
    }
}

```

2.3. Thuật toán Karp-Rabin

Trình bày thuật toán:

Trong thuật toán Brute Force, trường hợp xấu nhất là $O(m \times n)$. Khi m, n (n là độ dài văn bản, m là độ dài mẫu) lớn, việc so sánh sẽ mất nhiều thời gian. Thuật toán Karp-Rabin giải quyết vấn đề này bằng cách thay vì so sánh từng ký tự của cửa sổ (window) trong văn bản với từng ký tự trong mẫu, nó đưa cửa sổ và mẫu thành 1 giá trị băm để so sánh. Ngay khi thấy không khớp, nó sẽ chuyển sang cửa sổ mới.

Công thức tính giá trị băm:

$$\text{hash}(w[0..m-1]) = (w[0] \times 2^{m-1} + w[1] \times 2^{m-2} + \dots + w[m-1] \times 2^0) \bmod q$$

where q is a large number. Then,

$$\text{rehash}(a, b, h) = ((h - a \times 2^{m-1}) \times 2 + b) \bmod q.$$

Trong đó:

w là cửa sổ có độ dài m.

rehash(a, b, h) là giá trị băm của cửa sổ tiếp theo.

h là giá trị băm của cửa sổ trước đó.

a là ký tự đầu tiên của cửa sổ trước đó.

b là ký tự cuối cùng của cửa sổ tiếp theo.

Đánh giá độ phức tạp thuật toán:

- Pha tiền xử lý (pha khởi tạo hàm băm) có độ phức tạp $O(m)$.
- Pha tìm kiếm có độ phức tạp $O(m \times n)$.
- Mong đợi $O(m + n)$ ở thời gian chạy.

Kiểm nghiệm thuật toán:

Giả sử y là văn bản, x là mẫu.

Giá trị băm của mẫu là: $\text{hash}(\text{GCAGAGAG}) = 17597$.

Pha tìm kiếm:

Lần 1:

y

G	C	A	T	C	G	C	A
---	---	---	---	---	---	---	---

 G A G A G T A T A C A G T A C G

x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

$\text{hash}(y[0..7]) = 17819$

Lần 2:

y

G	C	A	T	C	G	C	A	G
---	---	---	---	---	---	---	---	---

 A G A G T A T A C A G T A C G

x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

$\text{hash}(y[1..8]) = 17533$

Lần 3:

y

G	C	A	T	C	G	C	A	G	A
---	---	---	---	---	---	---	---	---	---

 G A G T A T A C A G T A C G

x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

$\text{hash}(y[2..9]) = 17979$

Lần 4:

y

G	C	A	T	C	G	C	A	G	A	G
---	---	---	---	---	---	---	---	---	---	---

 A G T A T A C A G T A C G

x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

$\text{hash}(y[3..10]) = 19389$

Lần 5:

y G C A T C G C A G A G A G T A T A C A G T A C G
 x G C A G A G A G

$$\text{hash}(y[4..11]) = 17339$$

Lần 6:

y G C A T C G C A G A G A G T A T A C A G T A C G
1 2 3 4 5 6 7 8
 x G C A G A G A G

Lần 7:

y G C A T C G C A G A G A G T A T A C A G T A C G
 x G C A G A G A G

$$\text{hash}(y[6..13]) = 17102$$

Lần 8:

y G C A T C G C A G A G A G T A T A C A G T A C G
 x G C A G A G A G

$$\text{hash}(y[7..14]) = 17117$$

Lần 9:

y G C A T C G C A G A G A G T A T A C A G T A C G
 x G C A G A G A G

$$\text{hash}(y[8..15]) = 17678$$

Lần 10:

y G C A T C G C A G A G A G T A T A C A G T A C G
 x G C A G A G A G

Lần 11:

y G C A T C G C A G A G A G T A T A C A G T A C G
 x G C A G A G A G

$$\text{hash}(y[10..17]) = 17917$$

Lần 12:

y G C A T C G C A G A G A G T A T A C A G T A C G
 x G C A G A G A G

$$\text{hash}(y[11..18]) = 17723$$

Lần 13:

y G C A T C G C A G A G A G T A T A C A G T A C G
 x G C A G A G A G

$$\text{hash}(y[12..19]) = 18877$$

Lần 14:

y G C A T C G C A G A G A G T A T A C A G T A C G
 x G C A G A G A G

$$\text{hash}(y[13..20]) = 19662$$

Lần 15:

y G C A T C G C A G A G A G T A T A C A G T A C G
 x G C A G A G A G

$$\text{hash}(y[14..21]) = 17885$$

Lần 16:

y G C A T C G C A G A G A G T A T A C A G T A C G
 x G C A G A G A G

$$\text{hash}(y[15..22]) = 19197$$

Lần 17:

y G C A T C G C A G A G A G T A T A C A G T A C G
 x G C A G A G A G

$$\text{hash}(y[16..23]) = 16961$$

Ta thấy chỉ có lần 6 là giá trị băm của cửa sổ và của mẫu là trùng nhau. Sau đó ta thực hiện 8 lần so sánh các ký tự để đảm bảo rằng 2 chuỗi là thực sự giống nhau (vì 2 chuỗi khác nhau vẫn có thể có cùng 1 giá trị băm).

Trong ví dụ trên, thuật toán Karp-Rabin thực hiện 17 lần so sánh giá trị băm và 8 lần so sánh ký tự.

Lập trình theo thuật toán:

```
int memcmp(char *x, char *y, int m){
    int i;
```

```

    for(i = 0; i<m && x[i] == y[i]; ++i);
    if(i >= m) return 0;
    else return 1;
}

#define REHASH(a, b, h, d) (((h) - (a)*d) << 1) + (b))

void KR(char *x, int m, char *y, int n){
    int d, hx, hy, i, j;

    //preprocessing
    //Tính d = 2^(m-1) voi phép dịch trái. d<<1 ứng với dx2.
    for(d = i = 1; i < m; ++i)
        d = (d << 1);

    //khởi tạo hx, hy
    for(hx = hy = i = 0; i < m; ++i){
        hx = ((hx << 1) + x[i]);
        hy = ((hy << 1) + y[i]);
    }

    //searching
    j = 0;
    while(j <= n-m){
        if(hx == hy && memcmp(x, y + j, m) == 0){
            printf("%d\n", j);
        }
        hy = REHASH(y[j], y[j + m], hy, d);
        ++j;
    }
}

```

2.4. Thuật toán Shift Or

Trình bày thuật toán:

- * Thuật toán Shift Or sử dụng kỹ thuật bitwise.
- * Sử dụng 2 loại bit véc tơ là S và R. Cả S và R đều có độ dài m (m là độ dài mẫu x).
- Có cả thấy n véc tơ R_j ($0 \leq j \leq n-1$, n là độ dài của văn bản).
- Có k véc tơ S_c (c là 1 chữ cái trong văn bản y).
- * Cách tính véc tơ S_c :

for $0 \leq i \leq m-1$, $S_c[i] = 0$ nếu $x[i] = c$.

Véc tơ S_c chỉ ra vị trí của chữ cái c trong mẫu x.

- * Cách tính véc tơ R_j :

$$R_j[i] = \begin{cases} 0 & \text{if } x[0..i] = y[j-i..j] \\ 1 & \text{otherwise} \end{cases}$$

$$R_{j+1} = \text{Shift}(R_j) \text{ Or } S_{y[j+1]}$$

* Sau khi tính được véc tơ R_j , kiểm tra nếu $R_j[m-1] = 0$ thì vị trí mà mẫu x được tìm thấy trong văn bản y là $j - m + 1$.

Đánh giá độ phức tạp thuật toán:

Giả sử độ dài của mẫu không lớn hơn kích thước từ nhớ máy tính.

- Độ phức tạp pha tiền xử lý là $O(m + \sigma)$.
- Độ phức tạp pha tìm kiếm là $O(n)$.

Kiểm nghiệm thuật toán:

Tính các véc tơ S_C :

	S_A	S_C	S_G	S_T
G	1	1	0	1
C	1	0	1	1
A	0	1	1	1
G	1	1	0	1
A	0	1	1	1
G	1	1	0	1
A	0	1	1	1
G	1	1	0	1

Tính các véc tơ R_j :

		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
		G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
0	G	0	1	1	1	1	0	1	1	0	1	0	1	0	1	1	1	1	1	0	1	1	1	0	
1	C	1	0	1	1	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	
2	A	1	1	0	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	
3	G	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	
4	A	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	
5	G	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	
6	A	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	
7	G	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	

Ta thấy $R_{12}[7] = 0$ có nghĩa rằng tìm thấy x ở vị trí $12 - 8 + 1 = 5$.

Lập trình theo thuật toán:

```

int preSo(char *x, int m, unsigned int S[]) {
    unsigned int j, lim;
    int i;

    for (i = 0; i < ASIZE; ++i)
        S[i] = ~0;
    for (lim = i = 0, j = 1; i < m; ++i, j <= 1) {
        S[x[i]] &= ~j;
        lim |= j;
    }
    lim = ~(lim>>1);
    return(lim);
}

void SO(char *x, int m, char *y, int n) {
    unsigned int lim, state;
    unsigned int S[ASIZE];
    int j;

    if (m > WORD)
        error("SO: Use pattern size <= word size");

    /* Preprocessing */
    lim = preSo(x, m, S);

    /* Searching */
    for (state = ~0, j = 0; j < n; ++j) {
        state = (state<<1) | S[y[j]];
        if (state < lim)
            OUTPUT(j - m + 1);
    }
}

```

2.5. Thuật toán Morris-Pratt

Trình bày thuật toán:

Thuật toán Morris-Pratt cải thiện (giảm thiểu) số lần dịch chuyển cửa sổ bằng cách ghi nhớ một số phần của văn bản (text) mà đã match với mẫu (pattern). Điều này giảm thiểu số lần so sánh ký tự của mẫu và văn bản, từ đó làm tăng tốc độ tìm kiếm.

Giả sử ta đang xét đến vị trí j của văn bản y .

Và $x[0..i-1] = y[j..i+j-1] = u$, và bắt đầu không khớp tại $y[i+j]$: $a = x[i] \neq y[i+j] = b$.

Định nghĩa khái niệm **border**:

Giả sử ta có chuỗi $x[0..i-1]$ (có độ dài i) và chuỗi x có độ dài m .

Khi đó ta xét prefix (phần đầu) của x xem có match với suffix (phần cuối) của $x[0..i-1]$ không. Ta nói **border** của $x[0..i-1]$ là phần prefix dài nhất của x mà match với suffix của $x[0..i-1]$.

Gọi $mpNext[i]$ là độ dài của border của $x[0..i-1]$ ($0 \leq i \leq m$).

Khi đó, lần dịch của số tiếp theo sẽ bắt đầu so sánh $c = x[mpNext[i]]$ với $y[i+j] = b$ mà không sợ bỏ sót kết quả nào.

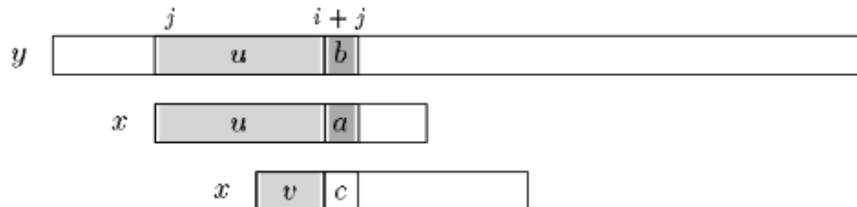


Figure 6.1 Shift in the Morris-Pratt algorithm: v is the border of u .

Thuật toán này thực hiện tối đa $2n - 1$ phép so sánh ký tự (n là độ dài của văn bản y). Trường hợp này văn bản y sẽ có tất cả n ký tự giống nhau và giống với ký tự đầu tiên của mẫu x nhưng không giống ký tự thứ 2 của x .

Đánh giá độ phức tạp thuật toán:

- Pha tiền xử lý: $O(m)$.
- Pha tìm kiếm: $O(n+m)$

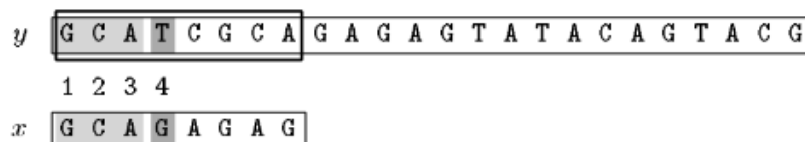
Kiểm nghiệm thuật toán:

* Pha tiền xử lý (khởi tạo các $mpNext[i]$).

i	0	1	2	3	4	5	6	7	8
$x[i]$	G	C	A	G	A	G	A	G	
$mpNext[i]$	-1	0	0	0	1	0	1	0	1

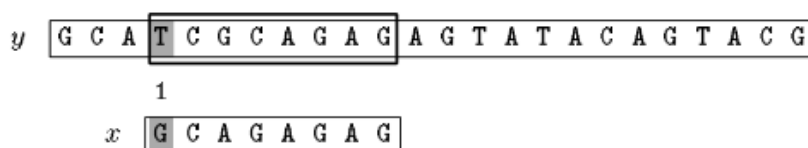
* Pha tìm kiếm.

Lần 1:



Shift by 3 ($i - mpNext[i] = 3 - 0$)

Lần 2:



Shift by 1 ($i - mpNext[i] = 0 - -1$)

Lần 3:

y

G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

1

 x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Shift by 1 ($i - mpNext[i] = 0 - -1$)

Lần 4:

y

G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

1 2 3 4 5 6 7 8

x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Shift by 7 ($i - mpNext[i] = 8 - 1$)

Lần 5:

y

G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

1

 x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Shift by 1 ($i - mpNext[i] = 1 - 0$)

Lần 6:

y

G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

1

x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Shift by 1 ($i - mpNext[i] = 0 - -1$)

Lần 7:

y

G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	C
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

1

 x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Shift by 1 ($i - mpNext[i] = 0 - -1$)

Lần 8:

y

G	C	A	T	C	G	C	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

1

x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Shift by 1 ($i - mpNext[i] = 0 - -1$)

Lần 9:



Shift by 1 ($i - mpNext[i] = 0 - -1$)

Ta thấy thuật toán Morris-Pratt thực hiện 19 lần so sánh ký tự trong ví dụ trên.

Lập trình theo thuật toán:

//pha tiền xử lý

```
void preMp(char *x, int m, int mpNext[]) {
    int i, j;

    i = 0;
    j = mpNext[0] = -1;
    while (i < m) {
        while (j > -1 && x[i] != x[j])
            j = mpNext[j];
        mpNext[++i] = ++j;
    }
}
```

//pha tìm kiếm

```
void MP(char *x, int m, char *y, int n) {
    int i, j, mpNext[XSIZE];

    /* Preprocessing */
    preMp(x, m, mpNext);

    /* Searching */
    i = j = 0;
    while (j < n) {
        while (i > -1 && x[i] != y[j])
            i = mpNext[i];
        i++;
        j++;
        if (i >= m) {
            OUTPUT(j - i);
            i = mpNext[i];
        }
    }
}
```

2.6. Thuật toán Knuth-Morris-Pratt.

Trình bày thuật toán:

Thuật toán này là một sự cải tiến của thuật toán Morris-Pratt.

Giả sử ta đang đặt cửa sổ tại vị trí j của văn bản y . Và $x[0..i-1] = y[j..i+j-1] = u$ và $a = x[i] \neq y[i+j] = b$.

Định nghĩa **tagged border**:

Tagged border là phần prefix của mẫu x mà match với phần suffix của u (giống với định nghĩa border của thuật toán Morris-Pratt).

Ta gọi $kmpNext[i]$ là độ dài của tagged border của $x[0..i-1]$ theo sau bởi ký tự c khác ký tự $x[i]$. Ngược lại thì $kmpNext[i] = -1$ ($0 < i \leq m$). Khi đó ta có thể tránh việc so sánh c với $y[i+j]$ (vì nếu $c = x[i]$ thì đương nhiên $c \neq y[i+j]$ nên ta không cần so sánh nữa).

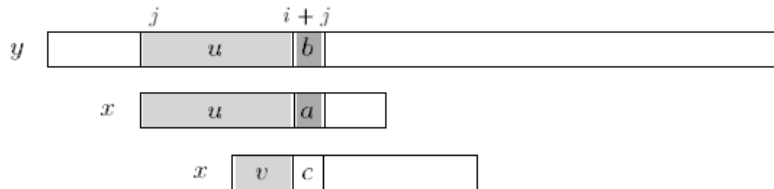


Figure 7.1 Shift in the Knuth-Morris-Pratt algorithm: v is a border of u and $a \neq c$.

Đánh giá độ phức tạp thuật toán:

- Pha tiền xử lý: $O(m)$.
- Pha tìm kiếm: $O(m + n)$.

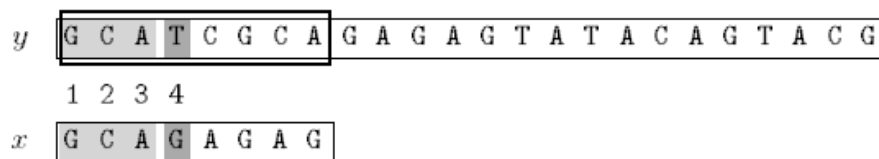
Kiểm nghiệm thuật toán:

* Pha tiền xử lý:

i	0	1	2	3	4	5	6	7	8
$x[i]$	G	C	A	G	A	G	A	G	
$kmpNext[i]$	-1	0	0	-1	1	-1	1	-1	1

* Pha tìm kiếm:

Lần 1:



Shift by 4 ($i - kmpNext[i] = 3 - (-1)$)

Lần 2:



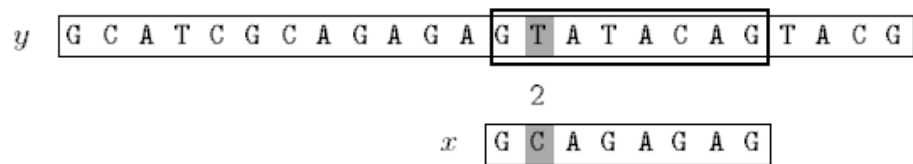
Shift by 1 ($i - kmpNext[i] = 0 - (-1)$)

Lần 3:



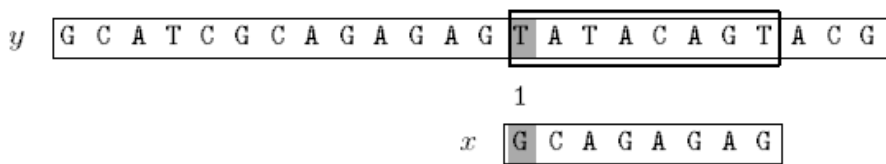
Shift by 7 ($i - kmpNext[i] = 8 - 1$)

Lần 4:



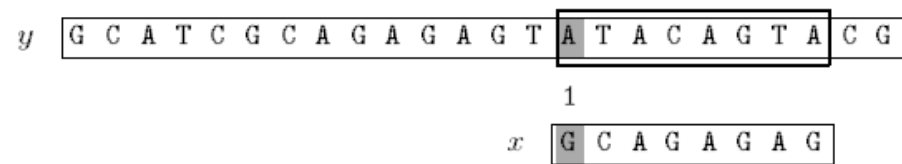
Shift by 1 ($i - kmpNext[i] = 1 - 0$)

Lần 5:



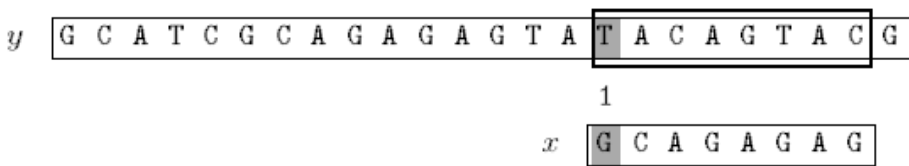
Shift by 1 ($i - kmpNext[i] = 0 - -1$)

Lần 6:



Shift by 1 ($i - kmpNext[i] = 0 - -1$)

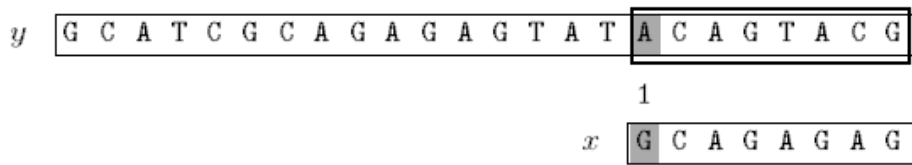
Lần 7:



Shift by 1 ($i - kmpNext[i] = 0 - -1$)

Lần 8:

Eighth attempt:



Shift by 1 ($i - kmpNext[i] = 0 - -1$)

Ta thấy thuật toán Knuth-Morris-Pratt thực hiện 18 lần so sánh ký tự trong ví dụ trên.

Lập trình theo thuật toán:

//pha tiền xử lý

```
void preKmp(char *x, int m, int kmpNext[]) {
    int i, j;

    i = 0;
    j = kmpNext[0] = -1;
    while (i < m) {
        while (j > -1 && x[i] != x[j])
            j = kmpNext[j];
        i++;
        j++;
        if (x[i] == x[j])
            kmpNext[i] = kmpNext[j];
        else
            kmpNext[i] = j;
    }
}
```

//pha tìm kiếm

```
void KMP(char *x, int m, char *y, int n) {
    int i, j, kmpNext[XSIZE];

    /* Preprocessing */
    preKmp(x, m, kmpNext);

    /* Searching */
    i = j = 0;
    while (j < n) {
        while (i > -1 && x[i] != y[j])
            i = kmpNext[i];
        i++;
        j++;
        if (i >= m) {
            OUTPUT(j - i);
            i = kmpNext[i];
        }
    }
}
```

2.7. Thuật toán Apostolico-Crochemore.

Trình bày thuật toán:

Thuật toán Apostolico-Crochemore sử dụng bảng kmpNext trong thuật toán Knuth-Morris-Pratt để tính độ dịch chuyển của cửa sổ.

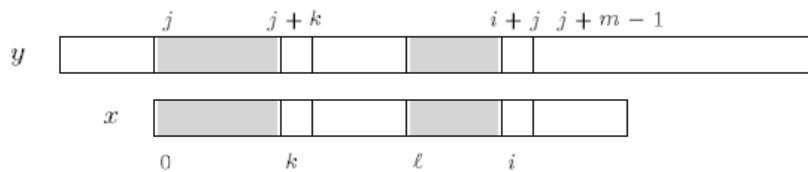
Đặt $l = 0$ nếu x là lũy thừa của 1 ký tự ($x = c^m$ hay nói cách khác x chứa m chữ cái giống nhau). Ngược lại, đặt $l =$ vị trí của ký tự đầu tiên trong x mà khác ký tự $x[0]$.

Mỗi lần thử, thuật toán so sánh theo thứ tự: $l, l+1, \dots, m-2, m-1, 0, 1, \dots, l-1$.

Trong pha tìm kiếm, ta luôn xem xét bộ ba (i, j, k) . Trong đó:

- Cửa sổ được đặt ở đoạn $y[j..j+m-1]$;
- $0 \leq k \leq l$ và $x[0..k-1] = y[j..j+k-1]$;
- $l \leq i < m$ và $x[l..i-1] = y[j+l..i+j-1]$.

Ban đầu khởi tạo $(i, j, k) = (l, 0, 0)$.



Cách tính bộ ba (i, j, k) tiếp theo dựa vào bộ ba trước đó:

- $i = l$
 Nếu $x[i] = y[i+j]$ thì bộ ba tiếp theo là $(i+1, j, k)$.
 Nếu $x[i] \neq y[i+j]$ thì bộ ba tiếp theo là $(l, j+1, \max\{0, k-1\})$.
- $l < i < m$
 Nếu $x[i] = y[i+j]$ thì bộ ba tiếp theo là $(i+1, j, k)$.
 Nếu $x[i] \neq y[i+j]$ thì ta có 2 trường hợp xảy ra phụ thuộc vào giá trị của $kmpNext[i]$:
 - Nếu $kmpNext[i] \leq l$ thì bộ ba tiếp theo là $(l, i + j - kmpNext[i], \max\{0, kmpNext[i]\})$.
 - Nếu $kmpNext[i] > l$ thì bộ ba tiếp theo là $(kmpNext[i], i + j - kmpNext[i], l)$.
- $i = m$
 Nếu $k < l$ và $x[k] = y[j+k]$ thì bộ ba tiếp theo là $(i, j, k+1)$.
 Ngược lại có 2 trường hợp: $k < l$ và $x[k] \neq y[j+k]$ hoặc $k = l$. Nếu $k = l$ thì thông báo tìm thấy một mẫu trong văn bản. Trong cả 2 trường hợp, bộ ba tiếp theo được tính như trong trường hợp $l < i < m$.

Đánh giá độ phức tạp thuật toán:

- Pha tiền xử lý: $O(m)$.
- Pha tìm kiếm: $O(n)$.

Kiểm nghiệm thuật toán:

i	0	1	2	3	4	5	6	7	8
$x[i]$	G	C	A	G	A	G	A	G	
$kmpNext[i]$	-1	0	0	-1	1	-1	1	-1	1

$\ell = 1$

First attempt:

y G C A T C G C A G A G A G T A T A C A G T A C G
 1 2 3
 x G C A G A G A G

Shift by 4 ($i - kmpNext[i] = 3 - -1$)

Second attempt:

y G C A T C G C A G A G A G T A T A C A G T A C G
 1
 x G C A G A G A G

Shift by 1 ($i - kmpNext[i] = 1 - 0$)

Third attempt:

y G C A T C G C A G A G A G T A T A C A G T A C G
 8 1 2 3 4 5 6 7
 x G C A G A G A G

Shift by 7 ($i - kmpNext[i] = 8 - 1$)

Fourth attempt:

y G C A T C G C A G A G A G T A T A C A G T A C G
 1
 x G C A G A G A G

Shift by 1 ($i - kmpNext[i] = 1 - 0$)

Fifth attempt:

y G C A T C G C A G A G A G T A T A C A G T A C G
 1
 x G C A G A G A G

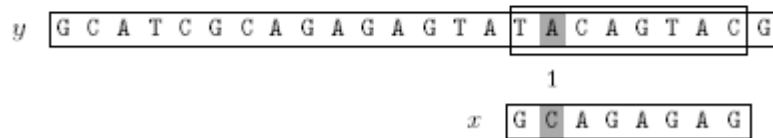
Shift by 1 ($i - kmpNext[i] = 0 - -1$)

Sixth attempt:



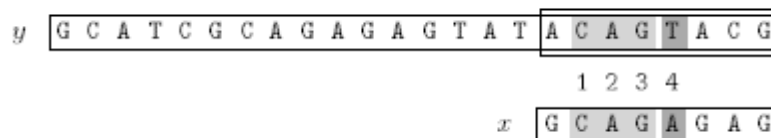
Shift by 1 ($i - kmpNext[i] = 0 - -1$)

Seventh attempt:



Shift by 1 ($i - kmpNext[i] = 0 - -1$)

Eighth attempt:



Shift by 3 ($i - kmpNext[i] = 4 - 1$)

Trong ví dụ trên thuật toán Apostolico-Crochemore thực hiện 20 phép so sánh ký tự.

Lập trình theo thuật toán:

```

void AXAMAC(char *x, int m, char *y, int n) {
    int i, j, k, ell, kmpNext[XSIZE];

    /* Preprocessing */
    preKmp(x, m, kmpNext);
    for (ell = 1; x[ell - 1] == x[ell]; ell++);
    if (ell == m)
        ell = 0;

    /* Searching */
    i = ell;
    j = k = 0;
    while (j <= n - m) {
        while (i < m && x[i] == y[i + j])
            ++i;
    }
}

```

```

    if (i >= m) {
        while (k < ell && x[k] == y[j + k])
            ++k;
        if (k >= ell)
            OUTPUT(j);
    }
    j += (i - kmpNext[i]);
    if (i == ell)
        k = MAX(0, k - 1);
    else
        if (kmpNext[i] <= ell) {
            k = MAX(0, kmpNext[i]);
            i = ell;
        }
        else {
            k = ell;
            i = kmpNext[i];
        }
    }
}

```

2.8. Thuật toán Not So Naïve

Trình bày thuật toán:

Trong pha tìm kiếm của thuật toán Not So Naïve, việc so sánh các ký tự của mẫu x được thực hiện theo thứ tự: $1, 2, \dots, m-2, m-1, 0$.

Mỗi lần thử (tức mỗi lần đặt cửa sổ vào đoạn $y[j..j+m-1]$ của văn bản y): nếu $x[0] = x[1]$ và $x[1] \neq y[j+1]$ (suy ra $x[0] \neq y[j+1]$) hoặc nếu $x[0] \neq x[1]$ và $x[1] = y[j+1]$ (suy ra $x[0] \neq y[j+1]$) thì cửa sổ được dịch 2 vị trí, ngược lại thì dịch 1.

Ta thấy thuật toán Not So Naïve gần giống với thuật toán Brute-Force nhưng có cải tiến một chút (vì vậy mới có tên là Not So Naive).

Đánh giá độ phức tạp thuật toán:

- Pha tiền xử lý: độ phức tạp hằng số
- Pha tìm kiếm: $O(m \times n)$.

Kiểm nghiệm thuật toán:

Từ mẫu x ta tính được $k = 1$, $\text{ell} = 2$ (k, ell theo code phía dưới).

Pha tìm kiếm:

First attempt:

y

G	C	A	T	C	G	C	A
---	---	---	---	---	---	---	---

 G A G A G T A T A C A G T A C G
 1 2 3
 x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Shift by 2

Second attempt:

y

G	C	A	T	C	G	C	A	G	A
---	---	---	---	---	---	---	---	---	---

 G A G T A T A C A G T A C G
 1
 x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Shift by 1

Third attempt:

y

G	C	A	T	C	G	C	A	G	A	G
---	---	---	---	---	---	---	---	---	---	---

 A G T A T A C A G T A C G
 1 2
 x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Shift by 2

Fourth attempt:

y

G	C	A	T	C	G	C	A	G	A	G
---	---	---	---	---	---	---	---	---	---	---

 T A T A C A G T A C G
 8 1 2 7 3 6 4 5
 x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Shift by 2

Fifth attempt:

y

G	C	A	T	C	G	C	A	G	A	G	A	G	T	A
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

 T A C A G T A C G
 1
 x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Shift by 1

Sixth attempt:

y

G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

 A C A G T A C G
 1
 x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Shift by 1

Seventh attempt:

y G C A T C G C A G A G A G T A T A C A G T A C G
1
 x G C A G A G A G

Shift by 1

Eighth attempt:

y G C A T C G C A G A G A G T A T A C A G T A C G
1
 x G C A G A G A G

Shift by 1

Ninth attempt:

y G C A T C G C A G A G A G T A T A C A G T A C G
1
 x G C A G A G A G

Shift by 1

Tenth attempt:

y G C A T C G C A G A G A G T A T A C A G T A C G
1
 x G C A G A G A G

Shift by 1

Eleventh attempt:

y G C A T C G C A G A G A G T A T A C A G T A C G
1
 x G C A G A G A G

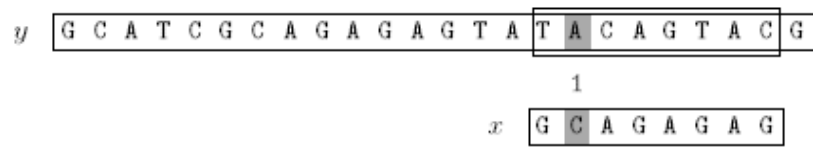
Shift by 1

Twelfth attempt:

y G C A T C G C A G A G A G T A T A C A G T A C G
1
 x G C A G A G A G

Shift by 1

Thirteenth attempt:



Shift by 1

Fourteenth attempt:



Shift by 2

Ta thấy thuật toán Not So Naïve thực hiện 27 phép so sánh ký tự trong ví dụ trên.

Lập trình theo thuật toán:

```
void NSN(char *x, int m, char *y, int n){
    int j, k, ell;

    //preprocessing
    if(x[0] == x[1]){
        k = 2;
        ell = 1;
    }else{
        k = 1;
        ell = 2;
    }

    //searching
    j = 0;
    while(j <= n-m){
        if(m > 1 && x[1] != y[j+1])
            j += k;
        else{
            if(m>1){
                if(memcmp(x + 2, y + j + 2, m - 2) == 0 && x[0]
== y[j])
                    printf("%d\n", j);
                j += ell;
            }
            else{
                if(x[0] == y[j])
                    printf("%d\n", j);
                ++j;
            }
        }
    }
}
```

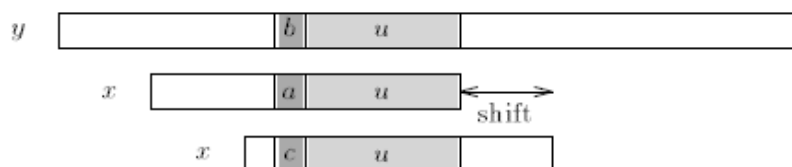
III. TÌM KIẾM MẪU TỪ PHẢI SANG TRÁI

3.1. Thuật toán Boyer-Moore

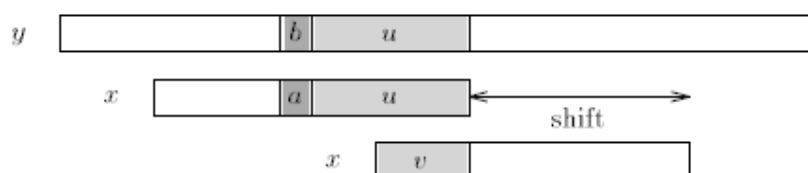
Trình bày thuật toán:

Thuật toán Boyer-Moore so sánh các ký tự của pattern x với văn bản y từ phải sang trái và dựa trên 2 kiểu dịch chuyển của sổ là: **good-suffix shift** và **bad-character shift**.

- **Good-suffix shift:** giả sử sự không khớp xuất hiện ở cặp $a = x[i]$ và $y[i+j] = b$ trong lần thử ở vị trí j của văn bản y . Do đó $x[i+1..m-1] = y[i+j+1..j+m-1] = u$ và $x[i] \neq y[i+j]$. Good-suffix shift đóng thẳng đoạn $y[i+j+1..j+m-1] = x[i+1..m-1] = u$ với 1 segment bên phải nhất của x sao cho segment đó $= u$ và ký tự ngay trước nó khác $x[i]$. Nếu không có một segment nào như vậy thì sẽ dịch cửa sổ sao cho đóng thẳng suffix dài nhất v của đoạn $y[i+j+1..j+m-1]$ với một prefix của x mà match với suffix đó.

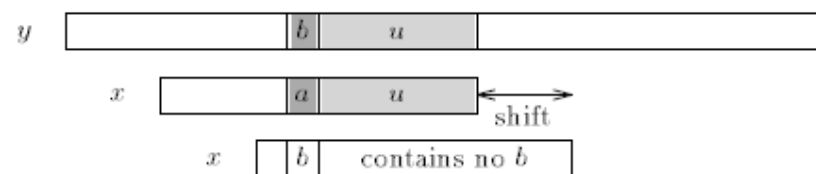


Hình trên biểu diễn good-suffix shift trong trường hợp tìm được 1 segment trong x mà ký tự c ngay trước nó khác a .

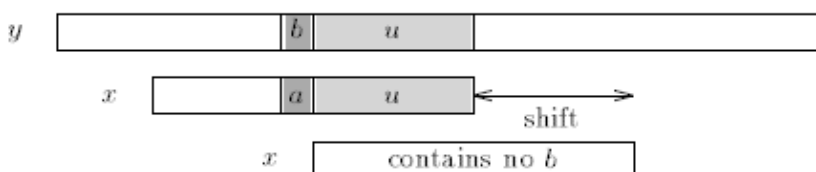


Hình trên biểu diễn good-suffix shift trong trường hợp không tìm được segment trong x thỏa mãn yêu cầu.

- **Bad-character shift:** Đóng ký tự $y[i+j] = b$ của văn bản y với 1 ký tự c bên phải nhất của đoạn $x[0..m-2]$ sao cho $c = b$. Nếu không tìm thấy ký tự $b = y[i+j]$ trong mẫu x thì ta dịch cửa sổ ra ngay sau ký tự b tức cửa sổ bắt đầu ở ký tự $y[i+j+1]$.



Hình trên biểu diễn bad-character shift trong trường hợp tìm thấy một ký tự b trong x thỏa yêu cầu.



Hình trên biểu diễn bad-character shift trong trường hợp không tìm thấy ký tự b trong x.

Như vậy với mỗi lần thử (mỗi cửa sổ) ta có 2 cách dịch chuyển cửa sổ là good-suffix shift và bad-character shift. Ngoài ra đối với bad-character shift, cửa sổ có thể bị dịch lùi lại (dịch sang trái). Ta sẽ chọn cách dịch chuyển mà dịch cửa sổ sang phải nhiều nhất.

Đánh giá độ phức tạp thuật toán:

- Pha tiền xử lý: $O(m + \sigma)$

- Pha tìm kiếm: $O(m \times n)$.

Kiểm nghiệm thuật toán:

- Pha tiền xử lý:

c	A	C	G	T
$bmBc[c]$	1	6	2	8

i	0	1	2	3	4	5	6	7
$x[i]$	G	C	A	G	A	G	A	G
$suff[i]$	1	0	0	2	0	4	0	8
$bmGs[i]$	7	7	7	2	7	4	7	1

- Pha tìm kiếm:

First attempt:

y G C A T C G C A G A G A G T A T A C A G T A C G

1

x G C A G A G A G

Shift by 1 ($bmGs[7] = bmBc[A] - 7 + 7$)

Second attempt:

y G C A T C G C A G A G A G T A T A C A G T A C G

3 2 1

x G C A G A G A G

Shift by 4 ($bmGs[5] = bmBc[C] - 7 + 5$)

Third attempt:

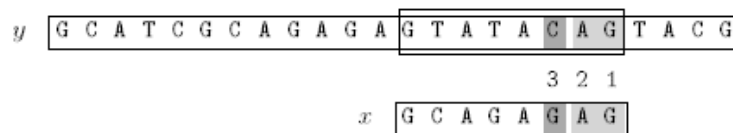
y G C A T C G C A G A G A G T A T A C A G T A C G

8 7 6 5 4 3 2 1

x G C A G A G A G

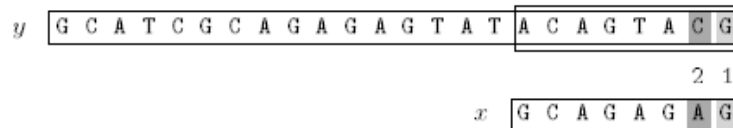
Shift by 7 ($bmGs[0]$)

Fourth attempt:



Shift by 4 ($bmGs[5] = bmBc[C] - 7 + 5$)

Fifth attempt:



Shift by 7 ($bmGs[6]$)

Ta thấy thuật toán Boyer-Moore thực hiện 17 phép so sánh ký tự trong ví dụ trên.

Lập trình theo thuật toán:

```
void preBmBc(char *x, int m, int bmBc[]) {
    int i;

    for (i = 0; i < ASIZE; ++i)
        bmBc[i] = m;
    for (i = 0; i < m - 1; ++i)
        bmBc[x[i]] = m - i - 1;
}

void suffixes(char *x, int m, int *suff) {
    int f, g, i;

    suff[m - 1] = m;

    g = m - 1;
    for (i = m - 2; i >= 0; --i) {
        if (i > g && suff[i + m - 1 - f] < i - g)
            suff[i] = suff[i + m - 1 - f];
        else {
            if (i < g)
                g = i;
            f = i;
            while (g >= 0 && x[g] == x[g + m - 1 - f])
                --g;
            suff[i] = f - g;
        }
    }
}
```



```

void preBmGs(char *x, int m, int bmGs[]) {
    int i, j, suff[XSIZE];

    suffixes(x, m, suff);

    for (i = 0; i < m; ++i)
        bmGs[i] = m;
    j = 0;
    for (i = m - 1; i >= -1; --i)
        if (i == -1 || suff[i] == i + 1)
            for (; j < m - 1 - i; ++j)
                if (bmGs[j] == m)
                    bmGs[j] = m - 1 - i;
    for (i = 0; i <= m - 2; ++i)
        bmGs[m - 1 - suff[i]] = m - 1 - i;
}

void BM(char *x, int m, char *y, int n) {
    int i, j, bmGs[XSIZE], bmBc[ASIZE];

    /* Preprocessing */
    preBmGs(x, m, bmGs);
    preBmBc(x, m, bmBc);

    /* Searching */
    j = 0;
    while (j <= n - m) {
        for (i = m - 1; i >= 0 && x[i] == y[i + j]; --i);
        if (i < 0) {
            OUTPUT(j);
            j += bmGs[0];
        }
        else
            j += MAX(bmGs[i], bmBc[y[i + j]] - m + 1 + i);
    }
}

```

3.2. Thuật toán Turbo-BM

Trình bày thuật toán:

Thuật toán Turbo-BM là sự cải tiến của thuật toán Boyer-Moore. Thuật toán này cốt yếu ở việc nó ghi nhớ đoạn text của y (chính là đoạn u trong thuật toán Boyer-Moore) mà match với suffix của x ở lần thử trước đó (chỉ khi một good-suffix shift được thực hiện).

Kỹ thuật này có 2 lợi thế:

- Nó có thể nhảy qua đoạn text đó (đoạn u).
- Nó cho phép thực hiện một **turbo-shift**.

Định nghĩa **turbo-shift**:

Turbo-shift chỉ có thể xuất hiện nếu ở lần thử hiện tại suffix của pattern mà match với text là ngắn hơn suffix ở lần thử trước đó. Trong trường hợp này chúng ta gọi u là suffix trước đó và v là suffix ở lần thử hiện tại. Gọi a và b là các ký tự gây ra sự không khớp (mismatch) trong lần thử hiện tại (a thuộc x và b thuộc y). Thì av là suffix của x và av cũng là suffix của u (do độ dài v bé hơn độ dài u). Hai ký tự a và b cách nhau khoảng cách p trong text, và suffix của x có độ dài $|uzv|$ có 1 đoạn độ dài $p = |zv|$. Vì u là **border** của $uzv \Rightarrow$ sẽ không tìm thấy x trong y khi ký tự b chạy sang trái 1 đoạn $|u| - |v| - 1$. Vậy độ dịch nhỏ nhất có thể có độ dài là $|u| - |v|$, ta gọi nó là **turbo-shift** (xem hình 15.1).

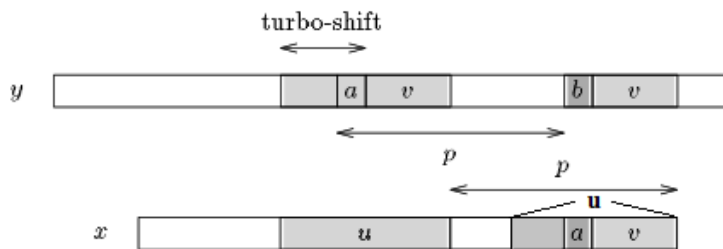


Figure 15.1 A turbo-shift can apply when $|v| < |u|$.

Vẫn xét trường hợp $|v| < |u|$, nếu độ dài của bad-character shift lớn hơn độ dài của good-suffix shift và turbo-shift thì độ dài dịch chuyển của số thực sự phải $\geq |u| + 1$. Thật vậy (xem hình 15.2), trong trường hợp này 2 ký tự c và d là khác nhau vì ta đang giả sử rằng lần dịch trước là good-suffix shift. Thì sau đó một sự dịch chuyển dài hơn turbo-shift nhưng ngắn hơn $|u| + 1$ sẽ dóng c và d vào vị trí với cùng một ký tự trong v , do vậy sẽ không tìm thấy một sự match nào (giả sử ký tự được dóng với c và d là e . Thì $e = d \Rightarrow e \neq c$ hoặc $e \neq d$. Cả 2 trường hợp đều có sự mismatch). Do đó, trường hợp này độ dài dịch chuyển thực sự ít nhất phải bằng $|u| + 1$.

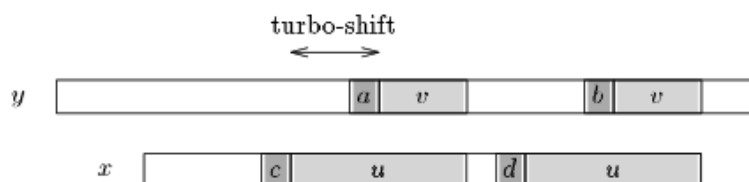


Figure 15.2 $c \neq d$ so they cannot be aligned with the same character in v .

Như vậy ta thấy rằng nếu $|v| \geq |u|$, ta so sánh good-suffix shift và bad-character shift để chọn ra cái tốt nhất. Ngược lại nếu $|v| < |u|$, ta so sánh turbo-shift, good-suffix shift, bad-character shift và $|u| + 1$ để chọn ra cái tốt nhất.

Đánh giá độ phức tạp thuật toán:

- Pha tiền xử lý: $O(m + \sigma)$.
- Pha tìm kiếm: $O(n)$.

Kiểm nghiệm thuật toán:

- * Pha tiền xử lý:


```

void TBM(char *x, int m, char *y, int n) {
    int bcShift, i, j, shift, u, v, turboShift,
        bmGs[XSIZE], bmBc[ASIZE];

    /* Preprocessing */
    preBmGs(x, m, bmGs);
    preBmBc(x, m, bmBc);

    /* Searching */
    j = u = 0;
    shift = m;
    while (j <= n - m) {
        i = m - 1;
        while (i >= 0 && x[i] == y[i + j]) {
            --i;
            if (u != 0 && i == m - 1 - shift)
                i -= u;
        }
        if (i < 0) {
            OUTPUT(j);
            shift = bmGs[0];
            u = m - shift;
        }
        else {
            v = m - 1 - i;
            turboShift = u - v;
            bcShift = bmBc[y[i + j]] - m + 1 + i;
            shift = MAX(turboShift, bcShift);
            shift = MAX(shift, bmGs[i]);
            if (shift == bmGs[i])
                u = MIN(m - shift, v);
            else {
                if (turboShift < bcShift)
                    shift = MAX(shift, u + 1);
                u = 0;
            }
        }
        j += shift;
    }
}

```

3.3. Thuật toán Quick Search

Trình bày thuật toán:

Là phiên bản đơn giản hóa của thuật toán Boyer-Moore.

Chỉ sử dụng bad-character shift.

Dễ dàng cài đặt.

Ta thấy rằng sau mỗi lần thử (tức cửa sổ được đặt ở đoạn $y[j..j+m-1]$ của text), độ dài dịch chuyển ít nhất phải bằng 1 $\Rightarrow$ vậy nên ký tự $y[j+m]$ đương nhiên sẽ được xét trong lần thử tiếp theo. Và vì vậy $y[j+m]$ có thể được sử dụng cho bad-character shift của lần thử hiện tại. Bad-character shift của thuật toán này được sửa đổi một chút như sau:

$$qsBc[c] = \begin{cases} \min(i + 1 : 0 \leq i < m) \text{ và } x[m - 1 - i] = c, \text{ nếu } c \text{ xuất hiện trong } x. \\ m + 1, \text{ nếu ngược lại.} \end{cases}$$

Như vậy trong mỗi lần thử tại đoạn $y[j..j+m-1]$, nếu không match với pattern x ta tìm bad-character shift. Với $\text{bad-character shift} = \text{qsBc}[y[j+m]]$.

Đánh giá độ phức tạp thuật toán:

- Pha tiền xử lý:

+ Độ phức tạp thời gian: $O(m + \sigma)$

+ Độ phức tạp bộ nhớ: $O(\sigma)$

- Pha tìm kiếm có độ phức tạp thời gian là $O(m \times n)$

Kiểm nghiệm thuật toán:

* Pha tiền xử lý:

a	A	C	G	T
$\text{qsBc}[a]$	2	7	1	9

* Pha tìm kiếm:

First attempt:

y	G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
	1	2	3	4																				
x	G	C	A	G	A	G	A	G																

Shift by 1 ($\text{qsBc}[G]$)

Second attempt:

y	G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
		1																						
x	G	C	A	G	A	G	A	G																

Shift by 2 ($\text{qsBc}[A]$)

Third attempt:

y	G	C	A	T	C	G	C	A	G	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
				1																						
x	G	C	A	G	A	G	A	G																		

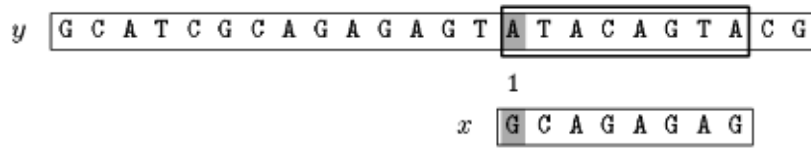
Shift by 2 ($\text{qsBc}[A]$)

Fourth attempt:

y	G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
						1	2	3	4	5	6	7	8											
x	G	C	A	G	A	G	A	G																

Shift by 9 ($\text{qsBc}[T]$)

Fifth attempt:



Shift by 7 ($qsBc[C]$)

Ta thấy thuật toán Quick Search thực hiện 15 phép so sánh ký tự trong ví dụ trên.

Lập trình theo thuật toán:

```
void preQsBc(char *x, int m, int qsBc[]){
    int i;

    for(i = 0; i<ASIZE; ++i)
        qsBc[i] = m + 1;

    for(i = 0; i < m; ++i)
        qsBc[x[i]] = m - i;
}

void QS(char *x, int m, char *y, int n){
    int j, qsBc[ASIZE];

    //preprocessing
    preQsBc(x, m, qsBc);

    //searching
    while(j <= n-m){
        if(memcmp(x, y + j, m) == 0)
            printf("%d\n", j);
        j += qsBc[y[j+m]]; //calculate the shift
    }
}
```

3.4. Thuật toán Tuned-Boyer-Moore

Trình bày thuật toán:

Tuned-Boyer-Moore là phiên bản đơn giản hóa của thuật toán Boyer-Moore. Dễ cài đặt và rất nhanh trong thực tế.

Phần tốn chi phí nhất trong các thuật toán đối sánh mẫu là kiểm tra xem liệu ký tự trong mẫu có khớp với ký tự trong cửa sổ không. Để tránh việc kiểm tra này nhiều lần, Tuned-Boyer-Moore sẽ dịch chuyển cửa sổ một số lần trước khi thực sự so sánh ký tự.

Thuật toán sử dụng bad-character shift để tìm ký tự $x[m-1]$ trong y . Thuật toán sẽ tiếp tục dịch chuyển cửa sổ cho đến khi tìm thấy $x[m-1]$ trong y . Thuật toán yêu cầu lưu giá trị của $bmBc[x[m-1]]$ vào biến *shift*, và sau đó đặt lại $bmBc[x[m-1]] = 0$. Khi $x[m-1]$ được tìm thấy trong y , $m-1$ ký tự còn lại được kiểm tra và sau đó dịch cửa sổ một đoạn *shift*.

Đánh giá độ phức tạp thuật toán: $O(m \times n)$

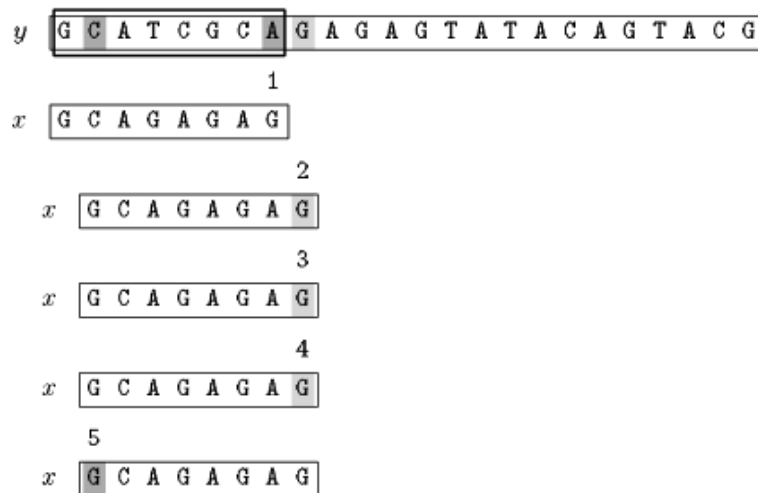
Kiểm nghiệm thuật toán:

a	A	C	G	T
$bmBc[a]$	1	6	0	8

$shift = 2$

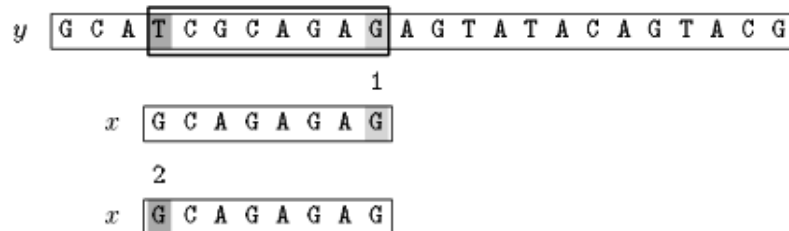
Searching phase

First attempt:



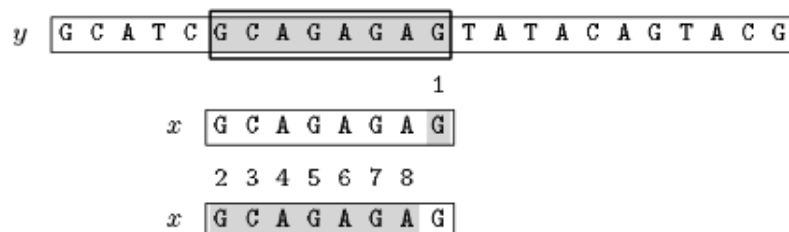
Shift by 2 ($shift$)

Second attempt:



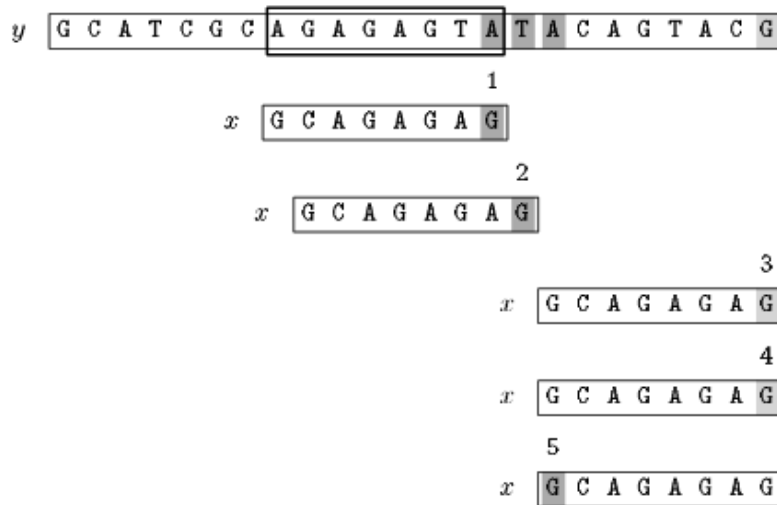
Shift by 2 ($shift$)

Third attempt:



Shift by 2 ($shift$)

Fourth attempt:



Shift by 2 (*shift*)

Ta thấy thuật toán Tuned-Boyer-Moore thực hiện 10 phép so sánh ký tự và 10 phép dịch chuyển trong ví dụ trên.

Lập trình theo thuật toán:

```
void TUNEDBM(char *x, int m, char *y, int n){
    int j, k, shift, bmBc[ASIZE];

    //preprocessing
    preBmBc(x, m, bmBc);

    shift = bmBc[x[m - 1]];
    bmBc[x[m - 1]] = 0;

    //searching
    j = 0;
    while(j <= n - m){
        k = bmBc[y[j + m - 1]];
        while(k != 0){
            j += k; k = bmBc[y[j + m - 1]];
            j += k; k = bmBc[y[j + m - 1]];
            j += k; k = bmBc[y[j + m - 1]];
            if(j > n - m) break;
        }

        if(memcmp(x, y + j, m - 1) == 0 && j <= n - m)
            printf("%d\n", j);
        j += shift;
    }
}
```

3.5. Thuật toán Zhu-Takaoka

Trình bày thuật toán:

Thuật toán Zhu-Takaoka là biến thể của thuật toán Boyer-moore. Thuật toán sử dụng 2 ký tự liên tiếp nhau để tính **bad-character shift**.

Trong pha tìm kiếm, các so sánh được thực hiện từ phải sang trái trong cửa sổ $y[j..j+m-1]$, và sự không khớp xuất hiện ở $x[m-k]$ và $y[j+m-k]$, và đoạn khớp là $x[m-k+1..m-1] = y[j+m-k+1..j+m-1]$. Thuật toán sẽ tính bad-character shift cho 2 ký tự cuối của cửa sổ là $y[j+m-2]$ và $y[j+m-1]$, đồng thời cũng tính cả good-suffix shift. Độ dịch lớn hơn sẽ được chọn để dịch cửa sổ ($\max(\text{bad-character shift}, \text{good-suffix shift})$).

Công thức tính bad-character shift cho 2 ký tự a,b.

For $a, b \in \Sigma$:

$$ztBc[a, b] = k \Leftrightarrow \begin{cases} k < m - 2 & \text{and } x[m - k .. m - k + 1] = ab \\ & \text{and } ab \text{ does not occur} \\ & \text{in } x[m - k + 2 .. m - 2] , \\ \text{or} \\ k = m - 1 & x[0] = b \text{ and } ab \text{ does not occur} \\ & \text{in } x[0 .. m - 2] , \\ \text{or} \\ k = m & x[0] \neq b \text{ and } ab \text{ does not occur} \\ & \text{in } x[0 .. m - 2] . \end{cases}$$

Đánh giá độ phức tạp thuật toán:

- Pha tiền xử lý: $O(m + \sigma^2)$.
- Pha tìm kiếm: $O(m \times n)$.

Kiểm nghiệm thuật toán:

- Pha tiền xử lý:

ztBc	A	C	G	T
A	8	8	2	8
C	5	8	7	8
G	1	6	7	8
T	8	8	7	8

i	0	1	2	3	4	5	6	7
$x[i]$	G	C	A	G	A	G	A	G
$bmGs[i]$	7	7	7	2	7	4	7	1

- Pha tìm kiếm:

First attempt:

y

G	C	A	T	C	G	C	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

1
 x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Shift by 5 ($ztBc[C][A]$)

Second attempt:

y

G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

8 7 6 5 4 3 2 1
 x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Shift by 7 ($bmGs[0]$)

Third attempt:

y

G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

3 2 1
 x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Shift by 4 ($bmGs[6]$)

Fourth attempt:

y

G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

2 1
 x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Shift by 7 ($bmGs[7] = ztBc[C][G]$)

Ta thấy thuật toán Zhu-Takaoka thực hiện 14 phép so sánh trong ví dụ trên.

Lập trình theo thuật toán:

```
void preZtBc(char *x, int m, int ztBc[ASIZE][ASIZE]) {
    int i, j;

    for (i = 0; i < ASIZE; ++i)
        for (j = 0; j < ASIZE; ++j)
            ztBc[i][j] = m;
    for (i = 0; i < ASIZE; ++i)
        ztBc[i][x[0]] = m - 1;
    for (i = 1; i < m - 1; ++i)
        ztBc[x[i - 1]][x[i]] = m - 1 - i;
}
```

```

void ZT(char *x, int m, char *y, int n) {
    int i, j, ztBc[ASIZE][ASIZE], bmGs[XSIZE];

    /* Preprocessing */
    preZtBc(x, m, ztBc);
    preBmGs(x, m, bmGs);

    /* Searching */
    j = 0;
    while (j <= n - m) {
        i = m - 1;
        while (i < m && x[i] == y[i + j])
            --i;
        if (i < 0) {
            OUTPUT(j);
            j += bmGs[0];
        }
        else
            j += MAX(bmGs[i],
                    ztBc[y[j + m - 2]][y[j + m - 1]]);
    }
}

```

3.6. Thuật toán Berry-Ravindran

Trình bày thuật toán:

Thuật toán Berry-Ravindran là thuật toán lai giữa 2 thuật toán Quick Search và Zhu-Takaoka. Thuật toán tính **bad-character shift** cho 2 ký tự liên tiếp nhau nằm ngay sau cửa sổ (là 2 ký tự $y[j+m]$ và $y[j+m+1]$).

Pha tiền xử lý thực hiện tính độ dịch bad-character shift cho cặp ký tự (a, b) với $a, b \in \Sigma$. Cặp (a, b) chính là cặp $(y[j+m], y[j+m+1])$. Ta đặt mẫu x vào giữa a, b (axb) và tìm sự xuất hiện bên phải nhất của ab trong $axb \Rightarrow$ từ đó tính được giá trị của bad-character shift cho (a, b) .

Cách tính bad-character shift cho (a, b) :

$$brBc[a, b] = \min \begin{cases} 1 & \text{nếu } x[m-1] = a, \\ m - i & \text{nếu } x[i]x[i+1] = ab \ (0 \leq i < m), \\ m + 1 & \text{nếu } x[0] = b, \\ m + 2 & \text{trong các trường hợp khác.} \end{cases}$$

Nếu cửa sổ hiện tại đang ở vị trí $y[j..j+m-1]$ thì bước dịch của sổ tiếp theo sẽ có độ dài là $brBc[y[j+m], y[j+m+1]]$.

Đánh giá độ phức tạp thuật toán:

- Pha tiền xử lý: $O(m + \sigma^2)$.
- Pha tìm kiếm: $O(m \times n)$.

Kiểm nghiệm thuật toán:

- Pha tiền xử lý:

$brBc$	A	C	G	T	*
A	10	10	2	10	10
C	7	10	9	10	10
G	1	1	1	1	1
T	10	10	9	10	10
*	10	10	9	10	10

The star (*) represents any character in $\Sigma \setminus \{A, C, G, T\}$.

- Pha tìm kiếm:

First attempt:

y G C A T C G C A G A G A G T A T A C A G T A C G
 1 2 3 4
 x G C A G A G A G

Shift by 1 ($brBc[G][A]$)

Second attempt:

y G C A T C G C A G A G A G T A T A C A G T A C G
 1
 x G C A G A G A G

Shift by 2 ($brBc[A][G]$)

Third attempt:

y G C A T C G C A G A G A G T A T A C A G T A C G
 1
 x G C A G A G A G

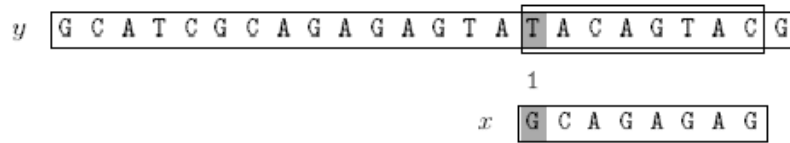
Shift by 2 ($brBc[A][G]$)

Fourth attempt:

y G C A T C G C A G A G A G T A T A C A G T A C G
 1 2 3 4 5 6 7 8
 x G C A G A G A G

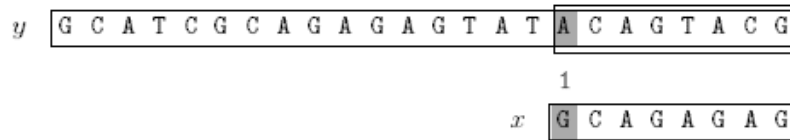
Shift by 10 ($brBc[T][A]$)

Fifth attempt:



Shift by 1 ($brBc[G][0]$)

Sixth attempt:



Shift by 10 ($brBc[0][0]$)

Ta thấy thuật toán Berry-Ravindran thực hiện 16 phép so sánh ký tự trong ví dụ trên.

Lập trình theo thuật toán:

```
void preBrBc(char *x, int m, int brBc[ASIZE][ASIZE]) {
    int a, b, i;

    for (a = 0; a < ASIZE; ++a)
        for (b = 0; b < ASIZE; ++b)
            brBc[a][b] = m + 2;
    for (a = 0; a < ASIZE; ++a)
        brBc[a][x[0]] = m + 1;
    for (i = 0; i < m - 1; ++i)
        brBc[x[i]][x[i + 1]] = m - i;
    for (a = 0; a < ASIZE; ++a)
        brBc[x[m - 1]][a] = 1;
}

void BR(char *x, int m, char *y, int n) {
    int j, brBc[ASIZE][ASIZE];

    /* Preprocessing */
    preBrBc(x, m, brBc);

    /* Searching */
    y[n + 1] = '\0';
    j = 0;
    while (j <= n - m) {
        if (memcmp(x, y + j, m) == 0)
            OUTPUT(j);
        j += brBc[y[j + m]][y[j + m + 1]];
    }
}
```

3.7. Thuật toán Apostolico-Giancarlo

Trình bày thuật toán:

Thuật toán Apostolico-Giancarlo là biến thể của thuật toán Boyer-Moore. Thuật toán Boyer-Moore khó để phân tích vì sau mỗi lần thử nó quên hết tất cả các ký tự đã khớp trước đó. Apostolico-Giancarlo ghi nhớ độ dài của suffix dài nhất của mẫu x sau mỗi lần thử. Thông tin này được lưu trong bảng **skip**.

Giả sử trong 1 lần thử ở vị trí nhỏ hơn j , thuật toán đã match một suffix độ dài k của x ở vị trí $i + j$ ($0 < i < m$) thì $\text{skip}[i+j] = k$. Nghĩa là $y[i+j-k+1..i+j] = x[m-1-k+1..m-1]$.

Đặt $\text{suff}[i]$ ($0 \leq i < m$) bằng độ dài suffix dài nhất của x kết thúc ở vị trí i trong x . Nghĩa là tồn tại $x[i-k+1..i] = x[m-1-k+1..m-1]$, trong đó $k = \text{suff}[i]$ (xem lại trong thuật toán Boyer-Moore).

Trong lần thử tại vị trí j , nếu đoạn text $y[i+j+1..j+m-1]$ đã khớp với mẫu x thì 4 trường hợp sau có thể xảy ra:

- TH1: $k > \text{suff}[i]$ và $\text{suff}[i] = i + 1$. Điều đó có nghĩa rằng ta đã tìm thấy một mẫu x trong y . Khi đó $\text{skip}[j+m-1] = m$. (Xem hình 16.1)

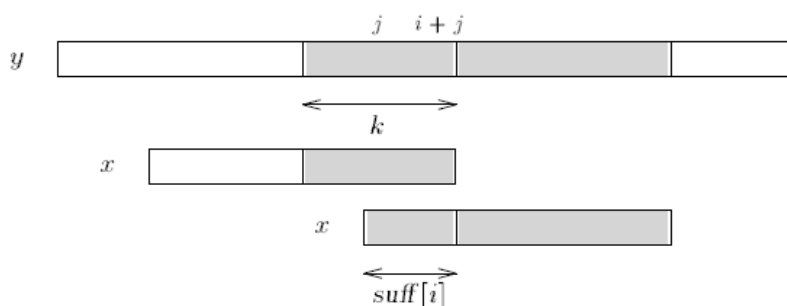


Figure 16.1 Case 1, $k > \text{suff}[i]$ and $\text{suff}[i] = i + 1$, an occurrence of x is found.

- TH2: $k > \text{suff}[i]$ và $\text{suff}[i] \leq i$. Điều đó có nghĩa là sự không khớp xảy ra giữa ký tự $x[i - \text{suff}[i]]$ và $y[i+j-\text{suff}[i]]$. Khi đó $\text{skip}[j+m-1] = m-1-i+\text{suff}[i]$ (Xem hình 16.2). Bước dịch được thực hiện sử dụng $\text{bmBc}[y[i+j-\text{suff}[i]]]$ và $\text{bmGs}[i-\text{suff}[i]+1]$.

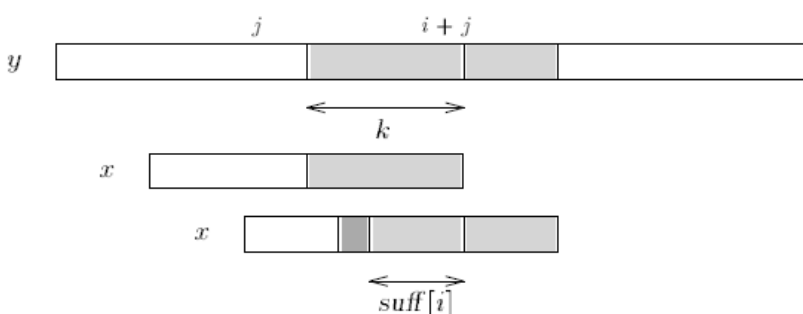


Figure 16.2 Case 2, $k > \text{suff}[i]$ and $\text{suff}[i] \leq i$, a mismatch occurs between $y[i+j-\text{suff}[i]]$ and $x[i-\text{suff}[i]]$.

- TH3: $k < \text{suff}[i]$, nghĩa là sự không khớp xảy ra giữa ký tự $x[i-k]$ và $y[i+j-k]$. Khi đó $\text{skip}[j+m-1] = m-1-i+k$ (Xem hình 16.3). Bước dịch được thực hiện sử dụng $\text{bmBc}[y[i+j-k]]$ và $\text{bmGs}[i-k+1]$.

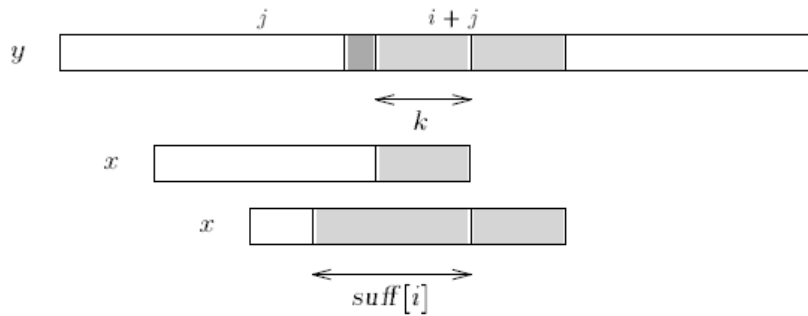


Figure 16.3 Case 3, $k < \text{suff}[i]$ a mismatch occurs between $y[i+j-k]$ and $x[i-k]$.

- TH4: $k = \text{suff}[i]$. Đây là trường hợp duy nhất bước nhảy phải nhảy qua đoạn $y[i+j-k+1..i+j]$ để so sánh 2 ký tự $y[i+j-k]$ và $x[i-k]$ (Xem hình 16.4)

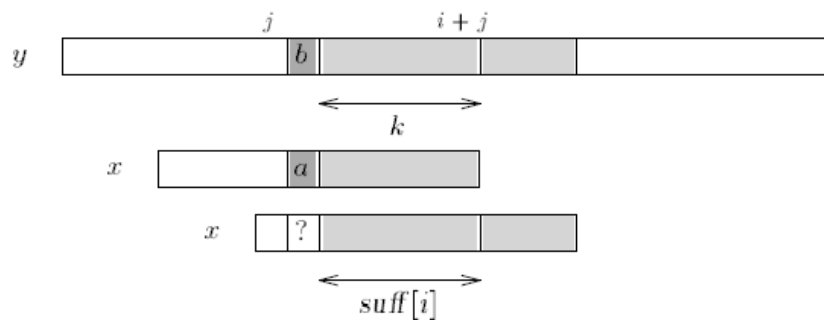


Figure 16.4 Case 4, $k = \text{suff}[i]$ and $a \neq b$.

Thuật toán Apostolico-Giancarlo sử dụng 2 cấu trúc dữ liệu:

- Một bảng skip được cập nhật ở cuối mỗi lần thử j theo cách sau:

$$\text{skip}[j+m-1] = \max\{k : x[m-k..m-1] = y[j+m-k..j+m-1]\}$$

- Một bảng suff sử dụng để tính bảng bmGs:

$$\text{for } 1 \leq i < m, \text{suff}[i] = \max\{k : x[i-k+1..i] = x[m-k, m-1]\}$$

Đánh giá độ phức tạp thuật toán:

- Pha tiền xử lý: $O(m + \sigma)$.

- Pha tìm kiếm: $O(n)$.

Kiểm nghiệm thuật toán:

- Pha tiền xử lý:

a	A	C	G	T
$\text{bmBc}[a]$	1	6	2	8

i	0	1	2	3	4	5	6	7
$x[i]$	G	C	A	G	A	G	A	G
$suff[i]$	1	0	0	2	0	4	0	8
$bmGs[i]$	7	7	7	2	7	4	7	1

- Pha tìm kiếm:

First attempt:

y G C A T C G C A G A G A G T A T A C A G T A C G
1
 x G C A G A G A G

Shift by 1 ($bmGs[7] = bmBc[A] - 7 + 7$)

Second attempt:

y G C A T C G C A G A G A G T A T A C A G T A C G
3 2 1
 x G C A G A G A G

Shift by 4 ($bmGs[5] = bmBc[C] - 7 + 5$)

Third attempt:

y G C A T C G C A G A G A G T A T A C A G T A C G
6 5 4 3 2 1
 x G C A G A G A G

Shift by 7 ($bmGs[0]$)

Fourth attempt:

y G C A T C G C A G A G A G T A T A C A G T A C G
3 2 1
 x G C A G A G A G

Shift by 4 ($bmGs[5] = bmBc[C] - 7 + 5$)

Fifth attempt:

y G C A T C G C A G A G A G T A T A C A G T A C G
2 1
 x G C A G A G A G

Shift by 7 ($bmGs[6]$)

Ta thấy thuật toán Apostolico-Giancarlo thực hiện 15 phép so sánh trong ví dụ trên.

Lập trình theo thuật toán:


```

void AG(char *x, int m, char *y, int n) {
    int i, j, k, s, shift,
        bmGs[XSIZE], skip[XSIZE], suff[XSIZE], bmBc[ASIZE];

    /* Preprocessing */
    preBmGs(x, m, bmGs, suff);
    preBmBc(x, m, bmBc);
    memset(skip, 0, m*sizeof(int));

    /* Searching */
    j = 0;
    while (j <= n - m) {
        i = m - 1;
        while (i >= 0) {

            k = skip[i];
            s = suff[i];
            if (k > 0)
                if (k > s) {
                    if (i + 1 == s)
                        i = (-1);
                    else
                        i -= s;
                    break;
                }
            else {
                i -= k;
                if (k < s)
                    break;
            }
        }
        else {
            if (x[i] == y[i + j])
                --i;
            else
                break;
        }
    }
    if (i < 0) {
        OUTPUT(j);
        skip[m - 1] = m;
        shift = bmGs[0];
    }
    else {
        skip[m - 1] = m - 1 - i;
        shift = MAX(bmGs[i], bmBc[y[i + j]] - m + 1 + i);
    }
    j += shift;
    memcpy(skip, skip + shift, (m - shift)*sizeof(int));
    memset(skip + m - shift, 0, shift*sizeof(int));
}

```

IV. TÌM KIẾM MẪU TỪ VỊ TRÍ XÁC ĐỊNH

4.1. Thuật toán Colussi

Trình bày thuật toán:

Thuật toán Colussi là sự cải tiến của thuật toán Knuth-Morris-Pratt.

Các vị trí của mẫu (pattern) được chia làm 2 tập con. Các vị trí trong tập con thứ nhất được duyệt từ trái sang phải và các vị trí trong tập con thứ 2 (các vị trí còn lại) được duyệt từ phải sang trái.

Mỗi lần thử (mỗi cửa sổ của văn bản y) gồm 2 pha (pha 1 luôn có, pha hai có thể có hoặc không):

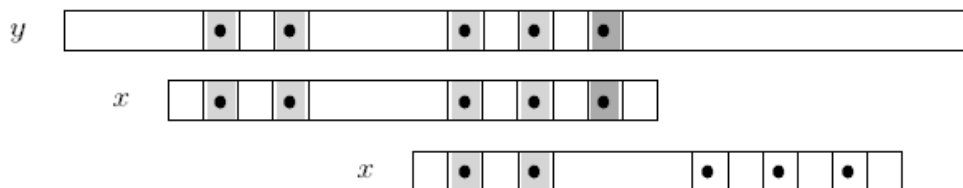
- Pha 1 thực hiện so sánh từ trái sang phải các ký tự trong text với các ký tự trong pattern mà $kmpNext$ có giá trị > -1 . Những vị trí này trong pattern được gọi là **noholes**.

- Pha 2 sẽ được thực hiện khi tất cả các so sánh trong pha 1 đều match. Nếu có 1 sự không khớp xảy ra trong pha 1 thuật toán sẽ dịch chuyển cửa sổ mà không thực hiện pha 2. Pha 2 so sánh các vị trí còn lại của pattern (các vị trí này gọi **holes**) từ phải sang trái. Nếu có 1 sự không khớp xảy ra, thuật toán sẽ dịch chuyển cửa sổ sang vị trí tiếp theo.

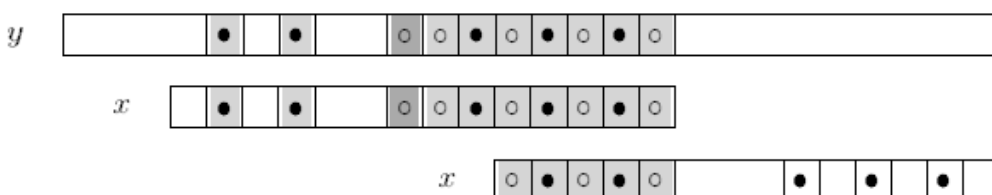
Chiến lược này có 2 ưu điểm:

- Khi 1 sự không khớp xảy ra trong pha 1, sau khi dịch chuyển cửa sổ thích hợp ta không cần phải so sánh các vị trí **noholes** đã được so sánh trong cửa sổ trước.

- Khi 1 sự không khớp (mismatch) xảy ra trong pha 2, nghĩa là có 1 đoạn suffix của pattern đã match với 1 đoạn của text. Sau khi dịch chuyển cửa sổ thích hợp, 1 prefix của pattern sẽ match với 1 suffix của pattern, mà suffix này lại match với 1 đoạn của text $\Rightarrow$ prefix cũng sẽ match với đoạn text đó $\Rightarrow$ khi dịch chuyển cửa sổ, ta không cần phải so sánh đoạn text đó nữa.



Hình trên là pha 1. Ta thấy khi dịch cửa sổ 2 vị trí noholes của pattern đã match với 2 vị trí trong text nên không cần so sánh nữa. Noholes được biểu diễn bằng hình tròn đen.



Hình trên là pha 2. Ta thấy tất cả vị trí noholes đã match với text. Ta thực hiện so sánh các vị trí holes từ phải sang. Khi dịch cửa sổ, ta không cần phải so sánh phần prefix của pattern với text nữa vì phần này đã match với text (phần prefix này gồm các noholes và holes mà đã match với text).

Đánh giá độ phức tạp thuật toán:

- Pha tiền xử lý: $O(m)$.
- Pha tìm kiếm: $O(n)$.

Kiểm nghiệm thuật toán:

* Pha tiền xử lý:

i	0	1	2	3	4	5	6	7	8
$x[i]$	G	C	A	G	A	G	A	G	
$kmpNext[i]$	-1	0	0	-1	1	-1	1	-1	1
$kmin[i]$	0	1	2	0	3	0	5	0	
$h[i]$	1	2	4	6	7	5	3	0	
$next[i]$	0	0	0	0	0	0	0	0	0
$shift[i]$	1	2	3	5	8	7	7	7	7
$hmax[i]$	0	1	2	4	4	6	6	8	8
$rmin[i]$	7	0	0	7	0	7	0	8	
$ndh0[i]$	0	0	1	2	2	3	3	4	

$$nd = 3$$

* Pha tìm kiếm:

First attempt:

y	G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
				1	2	3																		
x	G	C	A	G	A	G	A	G																

Shift by 3 ($shift[2]$)

Second attempt:

y	G	C	A	T	C	G	C	A	G	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
						1	2																			
x	G	C	A	G	A	G	A	G																		

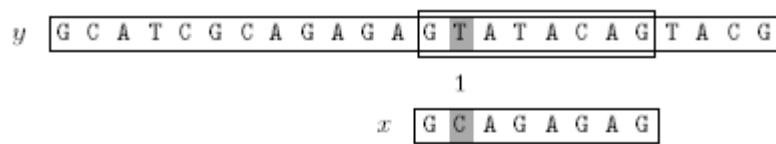
Shift by 2 ($shift[1]$)

Third attempt:

y	G	C	A	T	C	G	C	A	G	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
							8	1	2	7	3	6	4	5												
x	G	C	A	G	A	G	A	G																		

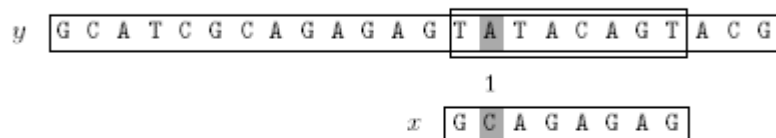
Shift by 7 ($shift[8]$)

Fourth attempt:



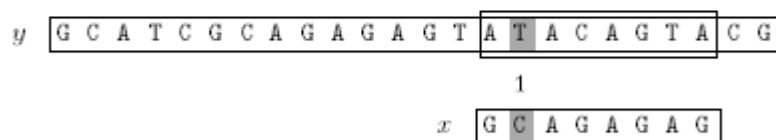
Shift by 1 ($shift[0]$)

Fifth attempt:



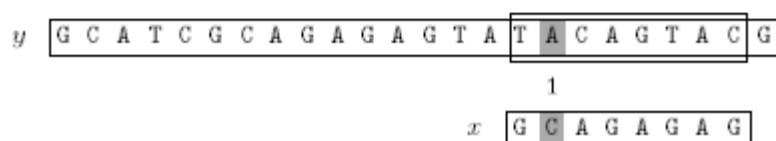
Shift by 1 ($shift[0]$)

Sixth attempt:



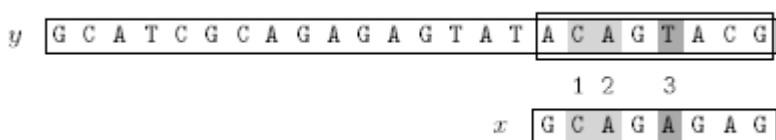
Shift by 1 ($shift[0]$)

Seventh attempt:



Shift by 1 ($shift[0]$)

Eighth attempt:



Shift by 3 ($shift[2]$)

Ta thấy thuật toán Colussi thực hiện 20 lần so sánh ký tự trong ví dụ trên.

Lập trình theo thuật toán:

//pha tiền xử lý

```

int preColussi(char *x, int m, int h[], int next[],
               int shift[]) {
    int i, k, nd, q, r, s;
    int hmax[XSIZE], kmin[XSIZE], nhd0[XSIZE], rmin[XSIZE];

    /* Computation of hmax */
    i = k = 1;
    do {
        while (x[i] == x[i - k])
            i++;
        hmax[k] = i;
        q = k + 1;
        while (hmax[q - k] + k < i) {
            hmax[q] = hmax[q - k] + k;
            q++;
        }
        k = q;
        if (k == i + 1)

            i = k;
    } while (k <= m);

    /* Computation of kmin */
    memset(kmin, 0, m*sizeof(int));
    for (i = m; i >= 1; --i)
        if (hmax[i] < m)
            kmin[hmax[i]] = i;

    /* Computation of rmin */
    for (i = m - 1; i >= 0; --i) {
        if (hmax[i + 1] == m)
            r = i + 1;
        if (kmin[i] == 0)
            rmin[i] = r;
        else
            rmin[i] = 0;
    }

    /* Computation of h */
    s = -1;
    r = m;
    for (i = 0; i < m; ++i)
        if (kmin[i] == 0)
            h[--r] = i;
        else
            h[++s] = i;
    nd = s;

```

```

/* Computation of shift */
for (i = 0; i <= nd; ++i)
    shift[i] = kmin[h[i]];
for (i = nd + 1; i < m; ++i)
    shift[i] = rmin[h[i]];
shift[m] = rmin[0];

/* Computation of nhd0 */
s = 0;
for (i = 0; i < m; ++i) {
    nhd0[i] = s;
    if (kmin[i] > 0)
        ++s;
}

/* Computation of next */
for (i = 0; i <= nd; ++i)
    next[i] = nhd0[h[i] - kmin[h[i]]];
for (i = nd + 1; i < m; ++i)
    next[i] = nhd0[m - rmin[h[i]]];
next[m] = nhd0[m - rmin[h[m - 1]]];

return(nd);
}

//pha tìm kiếm

void COLUSSI(char *x, int m, char *y, int n) {
    int i, j, last, nd,
        h[XSIZE], next[XSIZE], shift[XSIZE];

    /* Processing */
    nd = preColussi(x, m, h, next, shift);

    /* Searching */
    i = j = 0;
    last = -1;
    while (j <= n - m) {
        while (i < m && last < j + h[i] &&
            x[h[i]] == y[j + h[i]])
            i++;
        if (i >= m || last >= j + h[i]) {
            OUTPUT(j);
            i = m;
        }
        if (i > nd)
            last = j + m - 1;
        j += shift[i];
        i = next[i];
    }
}

```

4.2. Thuật toán Skip Search

Trình bày thuật toán:

Đối với mỗi ký tự trong bảng chữ cái, thuật toán Skip Search sử dụng một thùng (bucket) để chứa tất cả các vị trí của ký tự đó trong mẫu x. Khi một ký tự xuất hiện k lần trong mẫu x, sẽ có k vị trí tương ứng trong **bucket** của ký tự đó. Khi chuỗi y chứa ít ký tự hơn so với bảng chữ cái, sẽ có một số bucket rỗng.

Pha tiền xử lý của thuật toán Skip Search sẽ tính bucket cho tất cả các ký tự trong bảng chữ cái.

Đối với mỗi ký tự c thuộc bảng chữ cái, ta có:

$z[c] = \{i: 0 \leq i \leq m - 1 \text{ và } x[i] = c\}$. $z[c]$ chính là bucket của ký tự c.

Vòng lặp chính của pha tìm kiếm sẽ kiểm tra mọi ký tự thứ m, $y[j]$ (Do đó sẽ có n/m lần lặp). Đối với mỗi $y[j]$, nó sử dụng mỗi vị trí trong bucket $z[y[j]]$ để kiểm tra xem liệu tại mỗi vị trí trong $z[y[j]]$ thì có tìm được một x trong y không. Nó thực hiện so sánh x với y từ vị trí đầu tiên, lần lượt từng ký tự một cho đến khi gặp một sự không khớp hoặc tất cả đều khớp.

Đánh giá độ phức tạp thuật toán:

- Pha tiền xử lý: $O(m + \sigma)$.

- Pha tìm kiếm: $O(m \times n)$.

Kiểm nghiệm thuật toán:

- Pha tiền xử lý:

c	$z[c]$
A	(6, 4, 2)
C	(1)
G	(7, 5, 3, 0)
T	$\emptyset$

- Pha tìm kiếm:

First attempt:

y G C A T C G C A G A G A G T A T A C A G T A C G
 2 1
 x G C A G A G A G

y G C A T C G C A G A G A G A G T A T A C A G T A C G
 1
 x G C A G A G A G

y G C A T C G C A G A G A G T A T A C A G T A C G
 1 2 3 4 5 6 7 8
 x G C A G A G A G

Shift by 8

y

G	C	A	T	C	G	C	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

1

Third attempt:

y

G	C	A	T	C	G	C	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

2
1

 x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Lập trình theo thuật toán:

```

void SKIP(char *x, int m, char *y, int n) {
    int i, j;
    List ptr, z[ASIZE];

    /* Preprocessing */
    memset(z, NULL, ASIZE*sizeof(List));
    for (i = 0; i < m; ++i) {
        ptr = (List)malloc(sizeof(struct _cell));
        if (ptr == NULL)
            error("SKIP");
        ptr->element = i;
        ptr->next = z[x[i]];
        z[x[i]] = ptr;
    }

    /* Searching */
    for (j = m - 1; j < n; j += m)
        for (ptr = z[y[j]]; ptr != NULL; ptr = ptr->next)
            if (memcmp(x, y + j - ptr->element, m) == 0) {
                if (j - ptr->element <= n - m)
                    OUTPUT(j - ptr->element);
            }
        else
            break;
}

```

Trình bày thuật toán:

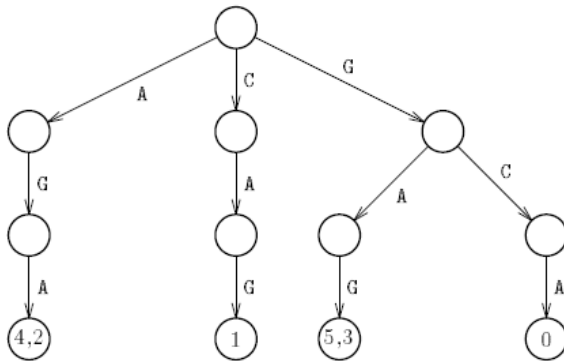
Pha tiền xử lý của thuật toán Alpha Skip Search xây dựng một cây $T(x)$. Trong đó các nhánh của cây biểu diễn cho các factor độ dài $L = \log_{\sigma} m$ xuất hiện trong x . Sau đó, có một bucket tại mỗi lá của $T(x)$ để chứa các vị trí xuất hiện của factor (tương ứng với nhánh) trong x .

Pha tìm kiếm xem xét bucket của factor $y[j..j+L-1]$ với tất cả $j = k \times (m - L + 1) - 1$, với k nằm trong đoạn $[1, \lfloor (n - L)/m \rfloor]$.

Đánh giá độ phức tạp thuật toán:

- Pha tiền xử lý: $O(m)$.
- Pha tìm kiếm: $O(m \times n)$.

Kiểm nghiệm thuật toán:

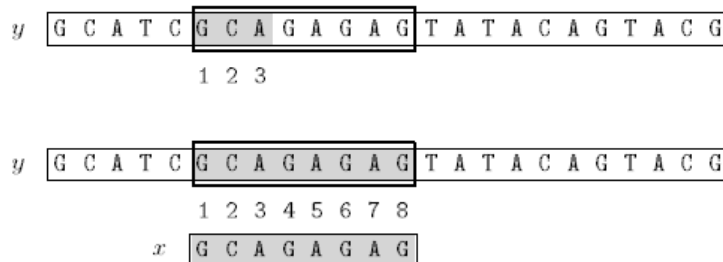


- Pha tiền xử lý:

u	$z[u]$
AGA	(4, 2)
CAG	(1)
GAG	(5, 3)
GCA	(0)

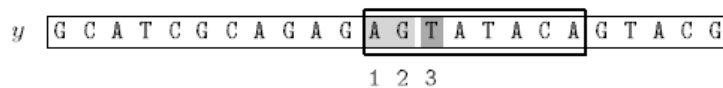
- Pha tìm kiếm:

First attempt:



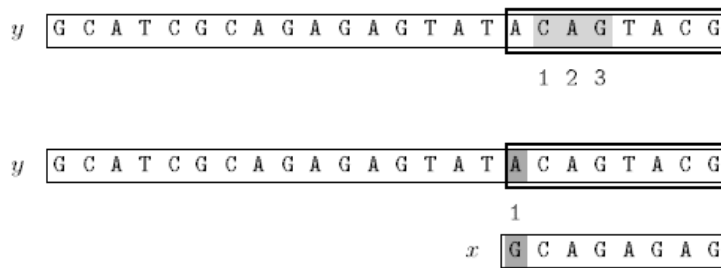
Shift by 6

Second attempt:



Shift by 6

Third attempt:



Ta thấy thuật toán Alpha Skip Search thực hiện 18 phép so sánh trong ví dụ trên.

Lập trình theo thuật toán:

```
List *z;

#define getZ(i) z[(i)]

void setZ(int node, int i) {
    List cell;

    cell = (List)malloc(sizeof(struct _cell));
    if (cell == NULL)
        error("ALPHASKIP/setZ");
    cell->element = i;
    cell->next = z[node];
    z[node] = cell;
}

/* Create the transition labelled by the
   character c from node node.
   Maintain the suffix links accordingly. */
int addNode(Graph trie, int art, int node, char c) {
    int childNode, suffixNode, suffixChildNode;

    childNode = newVertex(trie);
    setTarget(trie, node, c, childNode);
    suffixNode = getSuffixLink(trie, node);
    if (suffixNode == art)
        setSuffixLink(trie, childNode, node);
    else {
        suffixChildNode = getTarget(trie, suffixNode, c);
        if (suffixChildNode == UNDEFINED)
            suffixChildNode = addNode(trie, art,
                                      suffixNode, c);
        setSuffixLink(trie, childNode, suffixChildNode);
    }
    return(childNode);
}
```

```

void ALPHASKIP(char *x, int m, char *y, int n, int a) {
    int b, i, j, k, logM, temp, shift, size, pos;
    int art, childNode, node, root, lastNode;
    List current;
    Graph trie;

    logM = 0;
    temp = m;
    while (temp > a) {
        ++logM;
        temp /= a;
    }
    if (logM == 0) logM = 1;
    else if (logM > m/2) logM = m/2;

    /* Preprocessing */
    size = 2 + (2*m - logM + 1)*logM;
    trie = newTrie(size, size*ASIZE);
    z = (List *)calloc(size, sizeof(List));
    if (z == NULL)
        error("ALPHASKIP");

    root = getInitial(trie);
    art = newVertex(trie);
    setSuffixLink(trie, root, art);
    node = newVertex(trie);
    setTarget(trie, root, x[0], node);
    setSuffixLink(trie, node, root);
    for (i = 1; i < logM; ++i)
        node = addNode(trie, art, node, x[i]);
    pos = 0;
    setZ(node, pos);
    pos++;
    for (i = logM; i < m - 1; ++i) {
        node = getSuffixLink(trie, node);
        childNode = getTarget(trie, node, x[i]);
        if (childNode == UNDEFINED)
            node = addNode(trie, art, node, x[i]);
        else
            node = childNode;
        setZ(node, pos);
        pos++;
    }
    node = getSuffixLink(trie, node);
    childNode = getTarget(trie, node, x[i]);
}

```

```

if (childNode == UNDEFINED) {
    lastNode = newVertex(trie);
    setTarget(trie, node, x[m - 1], lastNode);
    node = lastNode;
}
else
    node = childNode;
setZ(node, pos);

/* Searching */
shift = m - logM + 1;
for (j = m + 1 - logM; j < n - logM; j += shift) {
    node = root;
    for (k = 0; node != UNDEFINED && k < logM; ++k)
        node = getTarget(trie, node, y[j + k]);
    if (node != UNDEFINED)
        for (current = getZ(node);
             current != NULL;
             current = current->next) {
            b = j - current->element;
            if (x[0] == y[b] &&
                memcmp(x + 1, y + b + 1, m - 1) == 0)
                OUTPUT(b);
        }
    }
    free(z);
}

```

V. TÌM KIẾM MẪU TỪ VỊ TRÍ BẤT KỲ

5.1. Thuật toán Horspool algorithm.

Trình bày thuật toán:

Thuật toán Horspool là phiên bản đơn giản hóa của thuật toán Boyer-Moore. Quy tắc dịch bad-character sử dụng trong thuật toán Boyer-Moore không hiệu quả đối với bảng chữ cái nhỏ, nhưng khi bảng chữ cái lớn so với độ dài của mẫu, như trường hợp với bảng ASCII và tìm kiếm thông thường trong một text editor thì nó trở nên rất hữu ích.

Horspool đề xuất chỉ sử dụng bad-character shift của ký tự ngoài cùng bên phải của số (ký tự $y[j+m-1]$) để tính độ dịch.

Đánh giá độ phức tạp thuật toán:

- Pha tiền xử lý: độ phức tạp thời gian là $O(m + \sigma)$ và độ phức tạp bộ nhớ là $O(\sigma)$.
- Pha tìm kiếm có độ phức tạp thời gian là $O(m \times n)$.

Kiểm nghiệm thuật toán:

- Pha tiền xử lý:

a	A	C	G	T
$bmBc[a]$	1	6	2	8

- Pha tìm kiếm:

First attempt:

y

G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

1
 x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Shift by 1 ($bmBc[A]$)

Second attempt:

y

G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

2 1
 x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Shift by 2 ($bmBc[G]$)

Third attempt:

y

G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

2 1
 x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Shift by 2 ($bmBc[G]$)

Fourth attempt:

y

G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

2 3 4 5 6 7 8 1
 x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Shift by 2 ($bmBc[G]$)

Fifth attempt:

y

G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

1
 x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Shift by 1 ($bmBc[A]$)

Sixth attempt:

y

G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

1
 x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Shift by 8 ($bmBc[T]$)

Seventh attempt:

y

G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

2 1
 x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Shift by 2 ($bmBc[G]$)

Thuật toán Horspool thực hiện 17 phép so sánh trong ví dụ trên.

Lập trình theo thuật toán:

```
void HORSPPOOL(char *x, int m, char *y, int n) {
    int j, bmBc[ASIZE];
    char c;

    /* Preprocessing */
    preBmBc(x, m, bmBc);

    /* Searching */
    j = 0;
    while (j <= n - m) {
        c = y[j + m - 1];
        if (x[m - 1] == c && memcmp(x, y + j, m - 1) == 0)
            OUTPUT(j);
        j += bmBc[c];
    }
}
```

5.2. Thuật toán Smith

Trình bày thuật toán:

Thuật toán sẽ giữ lại giá trị lớn nhất trong hai giá trị Horspool bad-character shift và Quick Search bad-character shift.

Smith nhận thấy rằng độ dịch của ký tự ngay sau ký tự ngoài cùng bên phải ($y[j+m]$) đôi khi ngắn hơn độ dịch của ký tự ngoài cùng bên phải cửa sổ ($y[j+m-1]$). Do đó, ông khuyên giữ lại giá trị lớn nhất trong 2 giá trị này.

Pha tiền xử lý của thuật toán Smith sẽ tính các giá trị Horspool bad-character shift và Quick Search bad-character shift.

Đánh giá độ phức tạp thuật toán:

- Pha tiền xử lý: độ phức tạp thời gian là $O(m + \sigma)$ và độ phức tạp bộ nhớ là $O(\sigma)$.
- Pha tìm kiếm có độ phức tạp thời gian là $O(m \times n)$.

Kiểm nghiệm thuật toán:

- Pha tiền xử lý:

a	A	C	G	T
$bmBc[a]$	1	6	2	8
$qsBc[a]$	2	7	1	9

- Pha tìm kiếm:

[illegible]Shift by 1 ($bmBc[A] = qsBc[G]$)

y

G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

1

x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

y

G	C	A	T	C	G	C	A	G	A	G
---	---	---	---	---	---	---	---	---	---	---

 A G T A T A C A G T A C G

1

x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

y

G	C	A	T	C	G	C	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

1 2 3 4 5 6 7 8

x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

y

G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

1

x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Lập trình theo thuật toán:

```

void SMITH(char *x, int m, char *y, int n) {
    int j, bmBc[ASIZE], qsBc[ASIZE];

    /* Preprocessing */
    preBmBc(x, m, bmBc);
    preQsBc(x, m, qsBc);

    /* Searching */
    j = 0;
    while (j <= n - m) {
        if (memcmp(x, y + j, m) == 0)
            OUTPUT(j);
        j += MAX(bmBc[y[j + m - 1]], qsBc[y[j + m]]);
    }
}

```

5.3. Thuật toán Raita

Trình bày thuật toán:

Trong mỗi lần thử, đầu tiên thuật toán so sánh ký tự cuối của mẫu và ký tự cuối của cửa sổ, nếu khớp nó so sánh ký tự đầu tiên của mẫu và ký tự đầu tiên của cửa sổ, nếu khớp nó so sánh ký tự giữa của mẫu và của cửa sổ. Nếu vẫn khớp nó mới so sánh các ký tự còn lại, bắt đầu từ ký tự thứ 2. Bước dịch được tính như thuật toán Horspool.

Đánh giá độ phức tạp thuật toán:

- Pha tiền xử lý: độ phức tạp thời gian là $O(m + \sigma)$ và độ phức tạp bộ nhớ là $O(\sigma)$.
- Pha tìm kiếm có độ phức tạp thời gian là $O(m \times n)$.

Kiểm nghiệm thuật toán:

- Pha tiền xử lý:

<i>a</i>	A	C	G	T
<i>bmBc[a]</i>	1	6	2	8

- Pha tìm kiếm:

First attempt:

y G C A T C G C A G A G A G T A T A C A G T A C G
1
 x G C A G A G A G

Shift by 1 (*bmBc[A]*)

y

G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
	2							1															

x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

third attempt:

y
G C A T C G C A G A G A G T A T A C A G T A C G

2 1

x
G C A G A G A G

fourth attempt:

y	G C A T C	G C A G A G A G	T A T A C A G T A C G
		2 4 5 6 3 8 9 1	
		7	
x	G C A G A G A G		

fifth attempt:

y

G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

1

x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

ixth attempt:

y	G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
																1								
x									G	C	A	G	A	G	A	G								

seventh attempt:

y	G C A T C G C A G A G A G T A T	A C A G T A C G
		2 1
x	G C A G A G A G	

thấy thuật toán Raita thực hiện 18 phép so sánh trong ví dụ
p trình theo thuật toán:

Lập trình theo thuật toán:

```

void RAITA(char *x, int m, char *y, int n) {
    int j, bmBc[ASIZE];
    char c, firstCh, *secondCh, middleCh, lastCh;

    /* Preprocessing */
    preBmBc(x, m, bmBc);
    firstCh = x[0];
    secondCh = x + 1;
    middleCh = x[m/2];
    lastCh = x[m - 1];

    /* Searching */
    j = 0;
    while (j <= n - m) {
        c = y[j + m - 1];
        if (lastCh == c && middleCh == y[j + m/2] &&
            firstCh == y[j] &&
            memcmp(secondCh, y + j + 1, m - 2) == 0)
            OUTPUT(j);
        j += bmBc[c];
    }
}

```